

Computer Science Department
Software Engineering & Business Analysis

Bachelor's Thesis

**Vakansio: A Personalized Job-
Matching Service**

Clean Architecture, LLM-Based Scoring, and Gold-Set
Evaluation on .NET 8

Yan Khadnevich

Supervisor

KSE, Volodymyr Skochko, vskochko@kse.org.ua

Expert

KSE, TBD, expert@kse.org.ua

Submission date

03 June 2026

Academic Integrity Statement

I, undersigned, hereby declare that this capstone project is the result of my own work.

- All ideas, data, figures and text from other authors have been clearly cited and listed in the bibliography.
- No part of this project has been submitted previously for academic credit in this or any other institution.
- All code, diagrams, and third-party materials are either my original work or are used with permission and properly referenced.
- I have not engaged in plagiarism or any form of academic dishonesty.
- Any assistance received (e.g. from peers, tutors, or online forums) is acknowledged in the acknowledgements section.

I understand that failure to comply with these declarations constitutes academic misconduct and may lead to disciplinary action.

Place, date Kyiv, 04.06.2026

Signature

Contents

Acknowledgements	1
Abstract	2
1 Introduction	3
1.1 Initial situation	3
1.2 Motivation	3
1.3 Objectives	4
1.4 Method	5
1.5 Structure of this thesis	5
2 Research	7
2.1 The Ukrainian job-search landscape	7
2.2 Existing job sites and what they offer	7
2.3 Legal and technical constraints on data collection	9
2.4 Related work in recommender systems	11
2.5 LLM-based evaluation frameworks	11
2.6 Gaps and our positioning	12
2.7 Functional and non-functional requirements	13
3 Design	15
3.1 Architectural drivers	15
3.2 Technology stack selection	16
3.3 Clean Architecture layering and SOLID mapping	17
3.4 Scoring pipeline design	19
3.5 Cache hierarchy	22
3.6 Frontend architecture and UX design	23
3.7 Deployment topology	25
3.8 Observability and security design	26
4 Implementation	29
4.1 Development methodology	29
4.2 Domain layer walkthrough	30
4.3 Application layer walkthrough	32
4.4 Infrastructure layer walkthrough	34
4.5 Frontend implementation details	36
4.6 Deployment, operations, and operating costs	38
4.7 Selected engineering narratives	41
5 Validation	43
5.1 Evaluation philosophy and methodology pivot	43
5.2 Layered evaluation architecture	44
5.3 Layers 1 and 2 — frozen normalisation baselines	46
5.4 Layer 3 — score as a calibration diagnostic	47
5.5 Layer 4 — reason-text quality and the prompt iteration cycle	49
5.6 Layer 5 — top-ten ranking quality (the primary endpoint)	52
5.7 End-to-end worked example	55
5.8 Threats to validity	57

6 Conclusion	60
6.1 Project summary	60
6.2 Comparison with the initial objectives	60
6.3 Encountered difficulties	61
6.4 Future perspectives	62
Glossary	64

Acknowledgements

The author thanks Volodymyr Skochko for supervising this capstone project and for the methodological guidance that shaped both the architectural commitments documented in the thesis and the framing of the evaluation chapter. The author also thanks the Kyiv School of Economics for the academic context within which the project was developed.

The author thanks family and friends for their support across the months during which the project was built and the thesis was written.

Abstract

A candidate searching for an information-technology job in Ukraine today has to visit several different job sites, each with its own listings, format, and search options, and decide one vacancy at a time whether the role is a fit. The work of comparing each vacancy to the candidate's own background is repeated by every candidate, every time, and across a full search accumulates into tens of evaluation hours under decision-fatigue conditions.

This thesis presents Vakansio, a production-deployed personalised job-matching service that addresses the problem at the candidate end. The system aggregates vacancies from seven external sources, normalises them and the candidate's curriculum vitae into structured representations, scores every candidate–vacancy pair through a deterministic seven-axis sub-score calculator refined by a Composite Large-Language-Model Judge, and presents the result with bilingual English and Ukrainian explanations. The implementation follows the Clean Architecture decomposition and the SOLID principles on a .NET 8 back-end with a React front-end, hosted on Amazon Web Services and reachable on the open internet.

The thesis contributes a five-layer evaluation methodology — curriculum-vitae normalisation, vacancy normalisation, score, reason text, and ranking — that follows the Holistic Evaluation of Language Models framework and applies methodology appropriate to each layer's data type. The reason-text layer reports a significant improvement of the v6 production prompt over the v4 baseline (overall $\Delta = +0.30$, 95 per cent paired-bootstrap confidence interval $[+0.19, +0.40]$, $N \approx 391$ pairs). The ranking layer is reported as directionally non-significant on a gold set of fourteen curricula vitae ($\Delta \text{NDCG}@10 = -0.047$, 95 per cent paired-bootstrap confidence interval $[-0.168, +0.065]$); a post-hoc sensitivity check on the proposed calibration clamp shifts the headline to $+0.018$ without changing the inconclusive verdict at the conventional significance level. The methodological framework stands independent of the empirical verdict, and the conditions for resolving the open question are stated explicitly.

Keywords:

Job Matching, Large Language Models, Clean Architecture, .NET 8, Recommender Systems, LLM-as-Judge

1 | Introduction

1.1 Initial situation

A person looking for an IT job in Ukraine today has to visit many different websites. Each site keeps its own list of vacancies, in its own format, with its own search and filter options. To find a position that actually fits, the candidate goes through each site one by one and opens vacancy after vacancy.

For every vacancy, the candidate reads the description, the list of requirements, and the company information, and then decides whether the job fits them. The same set of questions has to be answered each time: whether the role matches their background, whether they have the listed skills, whether the seniority level is appropriate, whether their English is sufficient, and whether the salary range is acceptable. Reading and judging a single vacancy takes a few minutes. A typical search returns dozens, sometimes hundreds of postings, so the whole process can take many evenings of work before the candidate even has a shortlist of jobs to apply to.

None of the popular Ukrainian IT job sites do this work for the candidate. Vacancies are usually shown by date or in the order the site chooses, but no popular site tells the user, before they start reading, that one vacancy is a strong fit for them and another is not. The work of comparing each vacancy with their own CV is repeated by every candidate, every time they look for a new job.

1.2 Motivation

For most people, looking for a new job is one of the more stressful and time-consuming tasks they face during their career. It is rarely just an evening of clicking through listings. A candidate who is serious about finding a position that truly fits has to dedicate weeks of effort: opening dozens of vacancies a day, comparing each one against their own background, preparing tailored applications, and keeping track of which jobs they have already applied to.

The time cost is the most visible part. If a candidate spends three to five minutes carefully reading one vacancy and another minute or two deciding whether to apply, a single evening of browsing covers roughly twenty or thirty postings. Most candidates have a full-time job or studies in parallel, so this work has to happen on top of everything else. Across a full search, the candidate can accumulate thirty to fifty hours just on the evaluation phase, before counting the time spent on interviews, technical tasks, and follow-up emails.

The hidden cost is the cognitive load. Every vacancy requires the candidate to switch context — different company, different stack, different seniority level, different country, sometimes a different language. Doing this for hours, day after day, leads to decision fatigue [1]. By the end of an evening, vacancies start to blur together, and the candidate begins to make worse decisions or to skip listings they should have read more carefully. Some good matches are missed; some bad matches are pursued.

The emotional cost is also significant. A long job search without visible progress reduces motivation over time, and repeated rejection — or, more often, no reply at all — leads candidates to doubt their own qualifications. The longer the search takes, the more this affects focus and effort, which in turn slows the search further.

A tool that takes on part of this work — that reads each vacancy carefully, compares it against the candidate’s CV, and indicates in advance where to focus — therefore has clear value. Saving even half of the evaluation time, and surfacing the most promising vacancies first, would change job search from a lengthy and exhausting process into one that is much more manageable.

1.3 Objectives

The project pursues four explicit objectives. Each one is restated as a measurable goal in later chapters.

The first objective is to build a working public service. The system must be reachable on the open internet, support user registration and login, accept a curriculum vitae as a PDF upload, search for vacancies from several sources, and return a personalised list with verdicts and explanations. The deployed instance is available at dsus1dizgh006.cloudfront.net and supports real user accounts.

The second objective is to follow a disciplined software architecture. The code base has to be organised according to the Clean Architecture pattern proposed by Martin [2] and the five SOLID principles. This is not merely a stylistic choice: the layer separation has to be visible in the code — the Domain project must not reference the Infrastructure project, and abstractions must live in the layer that needs them. The SOLID principles must be traceable to concrete artefacts. Section 3 documents the mapping in detail.

The third objective is to produce match quality that can be measured. An informal impression is not enough — the thesis has to define a methodology that another engineer could repeat. The evaluation is performed on two levels. The primary level compares the deterministic field-by-field outputs of the pipeline against author-curated reference normalisations of CVs and vacancies, using mean absolute error and verdict agreement, computed by the evaluation tool that ships with the code base. The secondary level uses an LLM acting as judge to rate match quality on a held-out batch of CV–vacancy pairs; ranking-quality numbers derived from these ratings are reported alongside the primary results in Section 5.

The fourth objective is to make the cost of every LLM call transparent. Each call must be logged with the pipeline stage that issued it, the number of input and output tokens, and the computed dollar cost. The data is written to a PostgreSQL table and is queryable by an operator. Per-stage cost numbers and aggregate statistics appear in Section 5.

Several topics are explicitly out of scope. The service does not allow employers to post their own vacancies — vacancies are aggregated from external sources only. There is no employer-side feature for searching candidates. The deployment is single-tenant: there is no organisation account model. Finally, an external user study with multiple real candidates was not conducted; the evaluation is author-internal against the gold sets described in Section 5, and the implications of this choice are discussed there.

1.4 Method

The system is built as a four-layer .NET 8 back-end, a React and TypeScript front-end, and a small AWS deployment. The four layers are Domain, Application, Infrastructure, and API. The Domain layer contains entities, value objects, and pure scoring rules that do not depend on any external library. The Application layer orchestrates use cases such as searching, scoring, and returning results, following the Command-Query Responsibility Segregation pattern. The Infrastructure layer adapts external dependencies: the Gemini large language model, the PostgreSQL database, Amazon S3 for CV files, and seven scrapers, APIs, and RSS feeds that fetch vacancies from job sites. The API layer is a thin HTTP entry point: controllers, authentication, and routing.

The scoring approach is hybrid. Some parts of the question of candidate–vacancy fit can be computed precisely: whether the listed skills overlap, whether the seniority level matches, and how many years of experience the candidate has, among other measurable signals. Those parts are computed by deterministic C# code organised as seven sub-scores: skill, seniority, experience, role intent, domain, language, and education. Their weighted sum is the first numeric estimate of fit. Other parts of the judgement are harder to express as fixed rules — for example, whether the vacancy description asks for the same kind of work the candidate has actually done. For these parts, the system calls Gemini 2.5 Flash to act as a Composite Judge that refines the deterministic score against a per-domain rubric. A second batched LLM call generates short bilingual explanations in English and Ukrainian for the top results.

The infrastructure is hosted on Amazon Web Services. A single t3.micro EC2 instance runs the API container behind Caddy. The database is an Amazon RDS PostgreSQL instance. CV files live in an Amazon S3 bucket. The front-end bundle is served from a second S3 bucket through CloudFront.

Development was organised iteratively, around versioned releases of the scoring pipeline rather than fixed-length sprints. The methodology, coding standards, and quality-assurance approach are documented in Section 4.

This summary covers the high-level shape only; the architecture, the scoring pipeline, the technology choices, and the deployment story are documented in the chapters that follow.

1.5 Structure of this thesis

The remainder of the document is organised as follows.

Section 2 surveys the Ukrainian job-search landscape, the approaches of the main job sites, and related work in recommender systems and large-language-model evaluation. It also lists the functional and non-functional requirements that the Vakansio system has to satisfy.

Section 3 presents the architectural drivers, the technology stack with justification against alternatives, the four-layer Clean Architecture decomposition, the scoring pipeline and Composite Judge, the cache hierarchy, the user-experience approach, and the mapping of the SOLID principles onto concrete artefacts in the code base.

Section 4 walks layer by layer through the Domain, Application, Infrastructure, and front-end implementations, documents the deployment and operations story on AWS, and describes two engineering challenges encountered during development.

Section 5 details the evaluation methodology, the construction of the gold sets, the primary numerical results, the secondary LLM-as-judge analysis, the per-stage cost analysis, unit and integration testing, the production-state snapshot at the time of submission, and the threats to validity — including the absence of an external user study.

Section 6 revisits the four objectives stated in this introduction, recounts the difficulties encountered, and outlines future work.

2 | Research

2.1 The Ukrainian job-search landscape

The Ukrainian information-technology market is large enough to make manual job search time-consuming for candidates. At any given moment, tens of thousands of open positions are visible across the various Ukrainian job sites, with new vacancies appearing daily. Candidates looking for a position typically check several sites in parallel, because no single source covers the whole market.

The landscape is fragmented. Different sites serve different audiences. Some focus specifically on information-technology and product roles, some are general-purpose and cover every industry, some are international networks with a Ukrainian segment, and at least one is a meta-aggregator that re-publishes listings from other boards. Each site stores its own catalogue, requires its own account, and presents listings in its own format. A candidate cannot search across all sources from a single place — they have to repeat the same query, with the same intent, on each site separately.

Within each site, the ranking of returned results is rarely personalised to an externally uploaded CV. The default ordering is usually by date of posting, and the available filters are coarse — location, seniority, salary range, sometimes a fixed set of technology tags. Where a site does offer a personalised signal at all, it operates on the candidate's structured on-platform profile rather than on a free-form CV document, and the score is provided without an explanation of how it is derived.

These two factors — fragmentation across sources, and the absence of CV-driven personalised ranking with explanation within each source — together define the gap that this thesis attempts to address. A service that aggregates vacancies from multiple Ukrainian sources and ranks them against an uploaded CV, with a human-readable explanation, is not offered by any mainstream Ukrainian platform today.

2.2 Existing job sites and what they offer

This section briefly profiles the major sites that are relevant to a Ukrainian information-technology candidate. The same profiling vocabulary is used for each — target audience, format of listings, strengths, and weaknesses with respect to personalised ranking against a candidate's CV.

Djinni.co is an information-technology-focused board with a candidate-oriented design that covers engineering, quality assurance, product management, design, marketing, and human-resources roles. Candidates create a profile with a declared salary expectation, a technology stack, and a seniority level, and employers see anonymised candidate cards. The platform is well known in the Ukrainian developer community for transparent salary practices and for protecting candidate identity until the candidate chooses to reveal it. Its filters are precise within the information-technology domain — language stack, salary range, English level, country, employment type — but the default ordering of the result list is reverse-chronological, and no public ranking against an externally uploaded CV document is available.

DOU.ua is one of the oldest Ukrainian developer communities, with an integrated job board. Listings tend to be detailed and written by employers themselves, and the board provides a public RSS feed for automated consumption. The search interface offers keyword and category filters, but no personalised ranking is provided. The board is strongest as a passive notice-board for the community; it is not designed for high-volume candidate-driven search.

Robota.ua is a general-purpose Ukrainian job board covering every industry, from retail and manufacturing to white-collar and information technology. The volume of listings is large, the audience is broad, and the platform offers standard filters for industry, region, schedule, and salary. Robota.ua advertises an AI-driven match indicator that scores how well a vacancy fits the candidate's on-platform profile, but the score operates on the platform's own structured profile rather than on an externally uploaded CV document, and it is presented without an explanation of why a particular vacancy received the score it did.

Work.ua is the other large general-purpose Ukrainian job board, comparable in scale to Robota.ua and hosting the largest Ukrainian CV database by volume. Its filters and presentation are similar to Robota.ua. The site does not provide a personalised ranking that takes an externally uploaded CV document into account; the relevance signal is keyword presence and the candidate's selected filters.

LinkedIn is an international professional network with a Ukrainian segment. The platform has its own machine-learning ranking of returned job results, taking into account the candidate's existing profile, connections, and engagement on the platform. The ranking is influenced by the LinkedIn profile rather than by an uploaded CV document, which means a candidate who maintains a thin LinkedIn profile but a rich CV will not benefit from the matching. The platform also tightly restricts automated access; this is discussed in Section 2.3.

Jooble is a meta-aggregator that collects listings from many national job boards (including the ones above) and presents them in a single interface, with a link out to the original source. The strength of Jooble is the breadth of its coverage; its weakness is that it does not produce its own ranking against an uploaded CV — the user receives a long list of links from other sites, sorted by the same default signals (recency and location) as the originating sources.

Several smaller and niche boards (Grc.ua, Happy Monday, Lobby.ua, Jobs.ua) operate alongside the six profiled above; they follow the same general pattern and are not separately profiled here.

Across the sites surveyed, the common pattern is keyword-based search with date-based ordering and coarse filters. Where a personalised signal exists at all, it operates on the site's own structured profile rather than on an externally uploaded CV, and it is provided without an accompanying explanation. None of the surveyed sites ranks an externally uploaded CV against multi-source listings with a human-readable explanation of why a particular listing received its position. This is the specific gap that the Vakansio service is designed to fill, and is justified further as a requirement in Section 2.7.

2.3 Legal and technical constraints on data collection

The use of multiple external job sources raises two practical questions that have to be addressed before any commercial deployment can be considered: whether the data collection is legally defensible, and whether the chosen technical channel is sustainable in production. This section first surveys the legal landscape in Ukraine and the European Union, then summarises the per-source position with respect to commercial use, and finally outlines the alternative service providers that would be required for a market version of Vakansio.

Legal landscape. The biggest risk for a Ukrainian company aggregating job listings is neither criminal liability under Article 361 of the Criminal Code, which concerns unauthorised interference with the operation of information and telecommunication systems, nor the General Data Protection Regulation applied to vacancy text — the text itself, without personal identifiers, is not personal data. The most material risk is civil. The 2022 Law of Ukraine on Copyright and Related Rights (No. 2811-IX), in force since 1 January 2023, introduced an explicit *sui generis* database right, giving fifteen years of protection to databases in which the maker has made a substantial investment. Sites such as Work.ua and Robota.ua are likely to qualify, although no Ukrainian court has yet ruled on this in the context of job-board aggregation. The controlling European Union precedent for this kind of situation is the Court of Justice ruling in *CV-Online Latvia v. Melons* [3], in which the operator of a Latvian meta-search engine indexing job listings was found to “extract and re-utilise” the database of CV-Online; the Court held that such acts may be prohibited by the database maker only where they adversely affect the investment in obtaining, verifying, or presenting the database contents. The ruling is directly applicable to the situation of Vakansio.

A separate consideration applies where individual listings contain incidental personal data — for example, named recruiter contacts or sole-proprietor employer details. Any such data ingested through the prototype’s scraping channels is processed under the same legitimate-interest basis or stripped on ingestion. The collection of CV files uploaded by the candidate is governed by a different basis. CV files contain personal data and trigger the full set of obligations under the Ukrainian Law on the Protection of Personal Data and, where the candidate is in the European Union, the General Data Protection Regulation. These obligations include an explicit lawful basis for the processing, retention limits, and the candidate’s right to access and deletion. They are honoured by Vakansio through explicit consent at upload time, scope limitation (CVs are used solely for matching), and deletion on account closure.

Recent litigation against scrapers of large platforms provides relevant context. In December 2022 the *hiQ Labs v. LinkedIn* proceedings ended with a stipulated judgment under which hiQ paid USD 500,000 and accepted a permanent injunction. This outcome stood despite an earlier appellate ruling that scraping publicly accessible data is not a violation of the United States Computer Fraud and Abuse Act. In July 2025 the LinkedIn data provider Proxycurl shut down following a permanent injunction in *LinkedIn v. Proxycurl*. These outcomes indicate that, while scraping public data is generally not a criminal

matter, civil liability arising from breach of platform terms of service is real and has materialised against well-funded specialised providers.

Per-source status. The seven sources currently used by the Vakansio prototype have different statuses with respect to commercial use. Table 1 summarises each one, together with the channel that a commercial version of the product would have to adopt.

Source	Status for commercial use	Channel for product
DOU.ua	Permitted via official RSS feed	Keep current RSS channel
Jooble	Permitted via official REST API	Keep current API channel
Djinni.co	Negotiable through partnership	Partner API via direct outreach
Robota.ua	Restricted; aggregator access dis-allowed	Business-development partnership
Work.ua	Restricted by terms of service	Partnership through partner contacts
LinkedIn	Prohibited; high legal risk	Paid licensed data provider
Manual URL	User-driven, no policy issue	Keep as is

Table 1: Per-source status for commercial use and the channel a market version would adopt.

Paid replacement for LinkedIn. Direct scraping of LinkedIn is not a sustainable commercial channel; replacement is therefore required. Several paid providers offer LinkedIn job-listing data on more clearly licensed terms. Apify, Bright Data, and JSearch make the data available through scraping-as-a-service offerings at approximately USD 100–300 per month for the volume needed by a small commercial deployment; these providers outsource the technical collection but do not assume contractual liability for breaches of LinkedIn terms of service. Coresignal positions itself as a licensed data vendor and provides equivalent coverage at approximately USD 1,000 per month under a negotiated commercial agreement, which is the option offering the most clearly delineated legal basis for a commercial launch. The LinkedIn Talent Solutions partner programme is, in practice, not accessible to a pre-revenue startup.

Path to commercial deployment. The mix proposed for a market version of Vakansio is therefore as follows. The free legitimate channels — the DOU RSS feed and the Jooble API — remain unchanged. A new channel is added through the OLX Partner API, which the prototype does not currently use. A parallel applicant-tracking-system aggregation channel is built on top of the free public boards exposed by Greenhouse, Lever, Workable, and SmartRecruiters; this channel covers Ukrainian-relevant employers such as Grammarly, GitLab, MacPaw, Kyivstar, and Ajax Systems, and brings several thousand additional openings without per-listing cost. Direct scraping of Djinni, Robota.ua, and Work.ua is replaced by partnership-based access through the data-licensing routes described above. Direct scraping of LinkedIn is replaced by a licensed paid provider — Coresignal for a production launch on a clearly delineated legal basis, or Apify or Bright Data for a minimum-viable product.

The Vakansio implementation evaluated in this thesis is a capstone prototype. The scraping channels currently in use are appropriate for that purpose: they access only publicly visible listings, do not bypass authentication, respect robots.txt where it expresses an explicit position with respect to the public listing pages, and operate at human-level rates. Commercial deployment would replace those channels with the official, partnership, and paid options described above.

2.4 Related work in recommender systems

A recommender system is software that suggests items to a user — products in an online shop, films on a streaming service, or vacancies on a job board. Recommender systems have been studied since the mid-1990s, and there are three main families of approach [4].

The first family is **content-based**. The system looks at facts the user has shared about themselves — for Vakansio, the uploaded CV — and finds items whose own description matches those facts. If the CV says the candidate knows React and TypeScript and has three years of experience, a content-based recommender will rank highly the vacancies that ask for React, TypeScript, and three or more years of experience.

The second family is **collaborative filtering**. The system looks at what other users have done — which items they liked, viewed, or applied to — and recommends the items that similar users picked. For Vakansio, this would mean ranking a vacancy highly because other candidates with a profile similar to the current candidate’s applied to it.

The third family is **hybrid**. The system combines content-based matching and collaborative filtering and weighs the two signals together.

Vakansio uses the content-based family. There are three reasons. First, a candidate who signs up for the first time has no past behaviour for the system to learn from, which means a collaborative filter would have nothing to compare against. Second, the CV and the vacancy both contain structured fields. This makes attribute matching simple. Third, the candidate expects an explanation of why a vacancy was ranked highly, and content-based matching can give that explanation directly, because the match is described using attributes from the candidate’s CV.

A separate question is how to measure whether a ranked list is good. A standard answer in information retrieval is a metric called **normalised discounted cumulative gain** (NDCG). The idea is simple: items shown higher in the list have more impact on the user, so the metric gives more weight to relevance at the top of the list and less to relevance at the bottom. The variant NDCG@10 measures the quality of the first ten results. It considers both how relevant each result is and how high it appears in the list. NDCG was first defined by Järvelin and Kekäläinen [5]. In this thesis, NDCG@10 is computed in a separate offline analysis presented in Section 5; it is not part of the production scoring pipeline.

2.5 LLM-based evaluation frameworks

Evaluating the output of a large language model is harder than evaluating a classifier. The output is free-form text rather than a single label, and several different answers can all be

reasonable. Two recent lines of work address this difficulty and are relevant to Vakansio’s evaluation methodology.

The first is HELM — Holistic Evaluation of Language Models — published by a team at Stanford [6]. The main idea of HELM is to evaluate a language model on many separate metrics rather than a single score. Instead of asking whether the model is good in general, HELM asks how accurate it is on a given task, how robust it is to small input changes, how calibrated its confidence scores are, and how fair it is across different groups of inputs, among other dimensions. Each metric is computed separately, so that strengths and weaknesses become visible. Vakansio borrows this multi-metric idea: every stage of the pipeline — CV normalisation, vacancy normalisation, sub-score correctness, ranking quality, and reason quality — is evaluated on its own metric, instead of a single accuracy number that hides where the system fails. The full layered methodology is described in Section 5.

The second is LLM-as-Judge — using a strong language model as the evaluator for the output of another model — proposed by Zheng et al. [7] in the context of MT-Bench and Chatbot Arena. The argument is empirical: when human annotators and a strong language-model judge (GPT-4 in the original paper) are asked to compare the same pair of answers, they agree more than 80% of the time. This makes the language-model judge a cheaper substitute for human ratings, especially when the evaluation set is too large to be rated by hand. Vakansio uses the LLM-as-Judge pattern in two places. Inside the scoring pipeline, a Composite Judge step refines the deterministic scores against a per-domain rubric. Inside the evaluation, a separate large language model is asked to rate a held-out set of CV–vacancy pairs on an ordinal scale; those ratings serve as the ground truth against which the production ranking is compared in Section 5.

2.6 Gaps and our positioning

Sections Section 2.1 to Section 2.5 surveyed two bodies of work that are relevant to Vakansio: the existing Ukrainian job-search market on the product side, and the academic literature on recommender systems and language-model evaluation on the methodology side. This section pulls the two together and states explicitly what Vakansio is and how it relates to each.

The product-side gap is specific. None of the Ukrainian job sites offers a personalised ranking of multi-source vacancies against an externally uploaded CV, accompanied by a human-readable explanation of why each match was made. This is true for the information-technology-focused boards such as Djinni and DOU, for the general-purpose boards such as Robota.ua and Work.ua, and for the international platforms such as LinkedIn and Jooble. Where partial elements of this exist — for example Robota.ua’s AI match indicator over its on-platform profile, or LinkedIn’s ranking based on the LinkedIn profile and engagement signals — they are tied to a single source’s internal representation of the candidate, and they are not accompanied by an explanation. Vakansio is the service that fills this combined gap.

On the methodology side, Vakansio applies a specific combination of ideas from recent work. It is a content-based recommender (Section 2.4), the natural choice for a system that

has to deliver useful recommendations from the candidate’s very first session and that has to explain each recommendation in terms of the candidate’s own CV. The scoring pipeline computes seven deterministic sub-scores and refines them through a Composite Judge step that follows the LLM-as-Judge pattern (Section 2.5). The evaluation methodology described in Section 5 follows HELM’s multi-metric approach (Section 2.5) — each stage of the pipeline has its own metric — and uses an LLM-as-Judge to construct the held-out ratings against which ranking quality is measured.

The novelty of Vakansio is not in any single one of these elements. Content-based recommendation is well established, the LLM-as-Judge pattern has been published, and multi-metric evaluation is the standard direction of the field. The contribution of this thesis is the combination: a multi-source CV-driven matching service for the Ukrainian market, delivered with explanations, built on a hybrid scoring pipeline that combines deterministic sub-scores with an LLM judge, and evaluated with a layered methodology that reports each stage’s quality separately rather than rolling everything up into a single number.

2.7 Functional and non-functional requirements

The work in Sections Section 2.1 to Section 2.6 motivates a specific set of requirements that the Vakansio system must satisfy. The requirements are grouped into two families. The first family captures what the system must do, expressed as user-testable behaviour. The second family captures how well the system must do it, as properties that matter once the system is deployed.

Functional requirements. The system must:

1. Allow a candidate to register an account and log in.
2. Accept a curriculum vitae as a PDF upload and parse it into a structured representation.
3. Search for vacancies from several external sources in response to a keyword query.
4. Return a ranked list of vacancies, ordered by the system’s estimate of fit between the uploaded CV and each vacancy.
5. Show, for each returned vacancy, a numerical match score between zero and one, a verdict label (Strong, Partial, Weak, or Mismatch), the matched skills, the missing must-have skills, and a short bilingual explanation in English and Ukrainian.
6. Support filtering of returned results by source, seniority level, and work format.
7. Allow a candidate to save vacancies of interest to a personal application tracker and to update their application status.

Non-functional requirements. The system must satisfy the following properties:

1. **Public deployment.** The system is reachable on the open Internet at a stable URL, supports real user accounts, and stays up without manual restarts.
2. **Clean Architecture.** The source code is organised according to Clean Architecture, with the Domain project free of external dependencies, interfaces declared in the layer that uses them, and SOLID principles traceable to specific files in the code.
3. **Cost transparency.** Every large-language-model call is logged with its pipeline stage, input and output token counts, and computed cost, and an operator can query the data at any time.

4. **Layered evaluation.** The quality of the system is measurable on a per-stage basis — CV normalisation, vacancy normalisation, sub-score correctness, ranking quality, and reason quality — rather than as a single end-to-end score.
5. **Security.** Authentication uses JSON Web Tokens with BCrypt-hashed passwords, secrets are not stored in plain text in the code base, and uploaded CV files are deleted on account closure.
6. **Reproducibility.** Deployment is reproducible from the checked-in Dockerfile, docker-compose.production.yml, and Caddyfile, and the deterministic scoring rules are documented in sufficient detail that another engineer could rebuild and redeploy the system.

The chapters that follow show how each of these requirements is satisfied: Section 3 maps them onto architectural decisions, Section 4 documents the implementation, and Section 5 reports the measurements of the non-functional ones.

3 | Design

3.1 Architectural drivers

This chapter describes the design of the Vakansio system. Every major decision in the chapter traces back to one or more requirements from Section 2.7. The strongest drivers — the requirements that shape several design decisions at once — are listed below as a roadmap to the rest of the chapter.

The first driver is the combination of **multi-source aggregation**, **CV-driven ranking**, and **human-readable explanation**. It calls for pluggable source adapters, so that each external job board can be added or replaced in isolation. It also calls for a single centralised scoring pipeline through which every (CV, vacancy) pair passes, and for a structured output that carries the score, the verdict, the matched skills, the missing must-haves, and the bilingual reason text. The responses are described in the technology stack section (Section 3.2), the Clean Architecture layering (Section 3.3), and the scoring pipeline design (Section 3.4).

The second driver is **production deployment at predictable cost**. The system has to stay reachable on the open Internet without manual intervention. This drives containerisation of the back-end, the use of managed services for the database and object storage, a reverse proxy that handles TLS automatically, and a static-hosting model for the front-end. The deployment topology is documented in Section 3.7.

The third driver is **measurable layer separation**. SOLID compliance has to be visible in the code, not only claimed. This requires a strict four-project decomposition (Domain, Application, Infrastructure, API), interfaces declared in the layer that uses them, and a command/query split that keeps each handler single-purpose. The decomposition is covered in Section 3.3.

The fourth driver is **per-stage evaluability and cost transparency**. Each pipeline stage must produce a measurement and a cost record. This drives a modular pipeline whose stages can be evaluated independently, a request-scoped accumulator that gathers per-stage cost without coupling the layers, a persistent cost-ledger table for offline querying, and an offline evaluation harness separate from the production code path. The responses appear in the scoring pipeline design (Section 3.4) and the observability section (Section 3.8).

The fifth driver is **security and data protection**. Uploaded CV files and user accounts are subject to the Ukrainian Law on the Protection of Personal Data and, where applicable, the General Data Protection Regulation. The architectural responses are stateless authentication based on JSON Web Tokens, secret storage in the AWS Systems Manager Parameter Store rather than the source tree, startup guards that refuse to start the system without an enforced SSL connection string and a CORS allow-list, and a cascade-delete pathway for full account removal. These are described in Section 3.8.

The sections that follow turn each driver into a concrete architectural choice.

3.2 Technology stack selection

This section presents the major technology choices behind Vakansio. Each choice is justified against the alternatives that were considered and rejected, with reference to the requirements stated in Section 2.7.

Back-end runtime: .NET 8. The back-end was implemented in C# on the .NET 8 runtime. The alternatives considered were Node.js with TypeScript, Go, and Python with FastAPI. .NET 8 was chosen for three reasons. First, ASP.NET Core has a mature dependency-injection container, a configuration-binding system, and a middleware pipeline that align well with the Clean Architecture and Options patterns required by Section 2.7. Second, C# offers a strong static type system with nullable reference types, which makes the interface contracts at layer boundaries enforceable at compile time. Third, the author had practical experience with the .NET ecosystem, which kept the development pace realistic for a solo project.

Front-end framework: React and TypeScript on Vite. The front-end was built as a single-page application in React with TypeScript and bundled by Vite. The available alternatives included Vue 3 and a server-side rendered framework such as Next.js. React was chosen because it has the largest npm download share among single-page-application frameworks and a deep set of mature companion libraries, including Zustand for client-side state management, TanStack Query for server data fetching, and React Router for client-side routing — each of which is used by Vakansio. TypeScript was chosen for the same compile-time contract enforcement reasoning as on the back-end. Vite was chosen over Webpack-based alternatives for fast hot-module reloading during development and a clean production build with no **webpack.config.js** to maintain. A server-side rendered approach was rejected because the user interface is stateful and client-driven; server-side rendering would add operational cost without a search-engine-optimisation benefit on an authenticated application.

Database: PostgreSQL via Amazon RDS. Relational storage was selected over a document store. The available alternatives included MySQL and MongoDB. PostgreSQL was preferred for three reasons. First, it offers first-class JSON column types (**jsonb**), which allow normalised vacancy and CV representations to live alongside relational data without the operational complexity of a separate document database. Second, its rich SQL feature set (window functions, full-text search, advisory locks) covers the analytical queries that the cost-ledger and evaluation paths require. Third, Amazon’s managed offering through the Relational Database Service removes the operational overhead of running and backing up the database, which matters for a solo project. MySQL was set aside because it lacks native **jsonb** support, which the normalisation cache relies on. MongoDB was rejected because the data model does not require schema-less storage, and the strong-consistency guarantees of PostgreSQL simplify the cache-invalidation reasoning described in Section 3.5.

Large language model: Gemini 2.5 Flash. The CV normalisation, vacancy normalisation, Composite Judge, and reason-generation stages all call into Gemini 2.5 Flash. The alternatives considered were OpenAI’s GPT-4-class models, Anthropic’s Claude, the Groq inference service, and a locally hosted open-weight model. Gemini was chosen for the

combination of low per-token cost, native support for structured JSON output that simplifies parsing, and acceptable latency at the volumes seen by Vakansio. The Composite Judge in particular benefits from a model that can follow a long calibration rubric reliably and return scores within a strict numerical range. GPT-4 and Claude were considered competitive on quality but materially more expensive per request at the scale of the pipeline. A locally hosted model was rejected because the t3.micro deployment target does not have enough memory or compute to serve a model with comparable instruction-following quality.

Cloud platform: Amazon Web Services. The system is hosted on Amazon Web Services. The available alternatives included Google Cloud Platform, Microsoft Azure, and DigitalOcean. The decisive factors were the author’s prior familiarity with the AWS console and command-line tooling, and the specifics of the AWS free-tier programme — a twelve-month allowance covering one t3.micro Elastic Compute Cloud instance, twenty gigabytes of Relational Database Service storage, and modest Simple Storage Service traffic — which together cover essentially the entire production cost envelope for a solo capstone project. The CloudFront edge network was an additional factor: it ensures low front-end latency for users across Ukraine without the need to operate a content-delivery network separately. Google Cloud and Azure offer comparable services and credits but introduce a learning-curve cost that was not warranted at the project’s scale.

Supporting libraries. Three further choices are worth noting briefly. Entity Framework Core is the object-relational mapper for PostgreSQL access; it was chosen over Dapper for its first-class migration tooling and its tight integration with the .NET dependency-injection container. MediatR provides the in-process request/response dispatcher that supports the Command-Query Responsibility Segregation pattern in the Application layer; it was preferred over a hand-rolled dispatcher for the ecosystem of behaviours (validation, logging, retry) available through its pipeline-behaviour abstraction. The BCrypt.Net-Next password-hashing library was chosen over a custom or weaker implementation because it implements the standard work-factor parameter and has a long track record of independent audits.

3.3 Clean Architecture layering and SOLID mapping

This section addresses the third architectural driver from Section 3.1. The requirements in Section 2.7 explicitly demand that SOLID compliance be visible in the code and that the layer separation be measurable rather than merely claimed. Clean Architecture [2] provides the structural answer, and SOLID provides the class-level answer; this section maps both onto the Vakansio code base.

The four layers. The back-end is organised as four .NET projects, named for the role they play. The Domain project contains the entities, value objects, enumerations, and pure scoring rules that describe the business of candidate–vacancy matching. It depends on no other project and on no third-party library — the file `Domain/Domain.csproj` declares zero package references. The Application project contains use cases, expressed as MediatR handlers, the options classes that configure them, and the abstractions through which the use cases call out to the rest of the system. It depends only on Domain. The Infrastructure project provides the concrete implementations of those abstractions

— the Entity Framework Core data context, the Gemini service clients, the AWS Simple Storage Service adapter, the Systems Manager Parameter Store reader, and the seven job-source adapters — and depends on Application (with Domain pulled in transitively). The API project is the thin HTTP entry layer: ASP.NET Core controllers, JSON Web Token middleware, the CORS configuration, options binding, and the startup file **Program.cs**. It depends on Application and Infrastructure.

The Dependency Rule. The decisive property of Clean Architecture is that source-code dependencies point inward only. Vakansio enforces this through the composition of project references: the Domain project is reference-free, the Application project references only Domain, the Infrastructure project references Application (and Domain transitively), and the API project references Application and Infrastructure. There is no path through which the Domain layer can be made to depend on, for example, the Gemini software development kit or Entity Framework Core. The compiler enforces the rule at build time. The same property makes the Domain layer trivially unit-testable, because there is no infrastructure to mock out.

Dependency Inversion in practice. The Application layer expresses what it needs through interfaces declared in its own folder (**Application/Common/Interfaces/**): **ICvFileStorage**, **ICostLogService**, **IScoringService**, **IBatchedJudgeService**, **IBatchedReasonService**, and others. Repository contracts owned by the entities themselves (for example **IJobVacancyRepository**) live in Domain instead. The Infrastructure layer provides the implementations. The composition root in **Program.cs** wires implementations to interfaces through the built-in .NET dependency-injection container. The handler classes in the Application layer never see a concrete class directly — every collaborator is injected through an interface. This makes the dependency direction explicit at every call site and allows entire infrastructure components to be substituted in tests or replaced in production (for example, swapping Gemini for OpenAI or for a local model would touch only the Infrastructure layer).

CQRS via MediatR. Use cases in the Application layer follow the Command-Query Responsibility Segregation pattern, with separate handler classes for read-only operations (under **Application/*/Queries/**) and state-changing operations (under **Application/*/Commands/**). Each handler implements **IRequestHandler<TRequest, TResponse>** and exposes exactly one **Handle** method. MediatR dispatches incoming requests to the right handler and supports cross-cutting concerns through its pipeline-behaviour abstraction. The pattern keeps individual handlers single-purpose: the largest handler, **GetAggregatedJobsV6Handler**, orchestrates the full search-and-score pipeline but contains no Gemini calls, no Entity Framework Core context, and no scraping logic; it composes those concerns through the interfaces injected into its constructor.

Versioning as cache-invalidation. A small but architecturally significant idiom is the use of version strings to drive cache invalidation. The Composite Judge prompt body carries a **BodyVersion** constant; the scoring pipeline exposes a composite **VersionWithJudge** string that concatenates the scoring version, the judge body version, and the model identifier; the persisted **ScoringCache** row stores the composite version it was generated against. When any underlying constant changes (because a prompt was edited or a weight was rebalanced), the resulting cache key changes, and every previously cached row

becomes unreachable by readers without any explicit invalidation pass. Cache correctness therefore does not depend on a separate operational procedure.

SOLID mapping. The five SOLID principles are mapped onto concrete artefacts in the Vakansio code base as shown in Table 2.

Principle	Concrete artefact in Vakansio
Single Responsibility	Domain/Scoring/Verdict.cs and Domain/Scoring/GenericGapFilter.cs each contain a single rule and no orchestration; GetAggregatedJobsV6Handler delegates all stages to injected services rather than executing them inline.
Open/Closed	Application/Common/Configuration/ScoringOptions.cs together with the IOptions<T> pattern makes tuning knobs such as SyncNormalizeTimeoutSeconds configurable through appsettings.json without recompilation.
Liskov Substitution	The scoring services depend on the interfaces IScoringService , IBatchedJudgeService , and IBatchedReasonService alternative implementations against another large-language-model provider would satisfy the same contracts with no consumer changes.
Interface Segregation	The three scoring interfaces are kept as separate single-purpose contracts rather than fused into one large interface; a class that implements IBatchedReasonService is not forced to also provide judging or scoring methods.
Dependency Inversion	Application handlers take only abstractions in their constructors; the composition root in Program.cs wires concrete implementations to those abstractions at startup, so the Application layer never imports an Infrastructure type directly.

Table 2: SOLID principles mapped onto concrete artefacts in the Vakansio code base.

Each row points to a file path that can be opened and read.

3.4 Scoring pipeline design

This section presents the design of the scoring pipeline — the part of Vakansio that converts a (CV, vacancy) pair into a numerical score, a verdict label, and a bilingual explanation. The pipeline is the system’s core technical contribution and the artefact that the validation in Section 5 evaluates.

Pipeline overview. Once the CV and the vacancies returned for the candidate’s query have been normalised, every (CV, vacancy) pair flows through the same sequence of stages: deterministic sub-scoring, an anti-flag penalty, an optional Composite Judge refinement, a safety cap, and finally a batched reason-generation step that produces the bilingual explanation. The shape of the pipeline is fixed; the stages are wired together by the scoring service implemented in **ScoringServiceV2.cs**.

The seven sub-score axes. The deterministic part of the score is a weighted sum of seven axis-specific sub-scores, each computed by a pure C# calculator under **Infrastructure/RelevancePipeline/V2/Scoring/SubScoreCalculators/**. The axes and their weights, taken from **ScoringServiceV2.cs** (lines 138–147), are shown in Table 3. The weights are not the result of an automated search — they are explicit design choices, and the most recent rebalance (Skill from 0.30 to 0.40, Language from 0.10 to 0.05, Education from 0.05 to 0.02) was motivated by the observation that the Language and Education scores are uniformly high across the candidate pool and therefore contribute little to discrimination, whereas the Skill score carries most of the signal that separates a good match from a poor one.

Axis	Weight	Note
Skill match	0.40	Primary discriminator (rebalanced from 0.30)
Role-intent match	0.15	
Seniority match	0.15	
Experience match	0.15	
Domain alignment	0.08	Modest reduction from 0.10
Language match	0.05	Was 0.10 — near-uniform high
Education match	0.02	Was 0.05 — near-uniform high
Total	1.00	

Table 3: Composite weights of the seven scoring axes, as configured in **ScoringServiceV2.cs:138–147**.

The deterministic linear score is the weighted sum

$$S_{\text{linear}} = \sum_{i=1}^7 w_i \cdot s_i, \quad \text{where } \sum_{i=1}^7 w_i = 1.00.$$

Anti-flag penalty. The deterministic linear score is then multiplied by an anti-flag penalty between zero and one. The penalty is produced by **AntiFlagEvaluator.cs** and triggers on conditions that suggest a candidate is structurally unsuited to the vacancy (for example, the vacancy requires on-site work in a city the candidate is unwilling to relocate to). The penalty is multiplicative rather than additive so that a single strong anti-flag can pull an otherwise strong score down decisively.

Role-family routing. Not every vacancy belongs to the same job family, and the relative weight of “what counts as a strong skill match” differs across families. Before the Composite Judge step, each vacancy is routed to one of a fixed set of role families — Product, Engineering, Data, Design, or a Generic fallback — by **KeywordRoleRouter.cs**. The router scores each candidate family by counting weighted matches of family-specific keywords in the vacancy title (with weight five) and description (with weight one); the family with the highest score above a confidence threshold of 0.30 wins, otherwise the routing falls through to Generic. The selected family in turn selects the Composite Judge calibration examples used in the next step.

Composite Judge with skip-band. The Composite Judge is a Gemini 2.5 Flash call that takes the (CV, vacancy, deterministic score, selected sub-scores) tuple and returns a refined score together with a brief justification. The judge prompt, defined in `JudgePromptCore.cs`, includes a calibration rubric that anchors specific score bands to qualitative descriptions (0.85–0.95 EXCELLENT, down to 0.20–0.25 MISMATCH) and a set of family-specific worked examples. The Composite Judge is gated by configuration (`ML:EnableCompositeJudge`) and includes a skip-band optimisation: pairs whose deterministic linear score falls outside the range $[0.30, 0.85]$ and have no active anti-flags bypass the Judge altogether, on the grounds that the deterministic signal is already extreme and a Judge call would only confirm it at the cost of a model invocation. The skip-band has been observed to reduce the number of Judge calls per query by approximately thirty to forty per cent.

Sub-score caps as a safety layer. After the Composite Judge step, the candidate score passes through `IScoringCapService`, which applies a floor of 0.20, a ceiling of 0.88, and a set of asymmetric caps that limit how high a score can go when a candidate has a clear seniority gap, a clear language gap, or a clear role-family mismatch. The caps act as a belt-and-braces safety layer: even if the language-model judge returns a misleadingly high score for an unsuitable match, the cap will pull it back into a range that reflects the structural mismatch.

Two-pass ranking. The full pipeline above is expensive to run on every candidate vacancy because the Composite Judge and the reason generation are model calls. To keep the per-request cost manageable, Vakansio runs a cheap pre-filter first. The deterministic linear score is computed for all returned vacancies, and only the top forty (configurable through `PreFilterTopN`) are passed on to the Composite Judge stage. Of those, the top thirty (configurable through `ReasonGenerationCap`) are passed to the batched reason-generation step. The two-pass shape is a deliberate cost–quality trade-off: the cheap pre-filter handles the easy decisions where the deterministic score is decisive, and the model calls are reserved for the cases where refinement is most likely to change the ranking.

Reason generation. For the top thirty pairs, a batched reason-generation step produces a structured JSON response that contains six text fields per pair — strengths, gaps, and recommendation, each in English and Ukrainian. The work is split into chunks of ten pairs per Gemini call, with up to three chunks dispatched in parallel. Each call uses `temperature=0.1` for predictability, validates the response against a strict schema, retries once if the schema check fails, and finally falls back to a deterministic template if both attempts fail. The bilingual JSON schema lets the front-end render the appropriate language without a second round trip.

Skill expansion and canonicalisation. Before any sub-score is computed, the raw skill tokens from both the CV and the vacancy are expanded and canonicalised against a global vocabulary. `SkillCanonicalizer.cs` maps synonyms (for example “React.js”, “ReactJS”, and “react”) onto a single canonical form, and the global-vocabulary path in `SkillVocabularyService` handles rare or compound terms that the static vocabulary does not cover, through a single batched Gemini call per request. The global-vocabulary path was introduced in version 6.3 to replace an earlier per-entity expander that produced many false-negative skill matches due to surface-form variation.

Evidence pipeline. The matched and missing skill chips that the front-end shows for each vacancy are not the raw output of the sub-score calculators. They pass through a four-step evidence pipeline — Build, Sanitise (deduplicate, strip Common European Framework of Reference for Languages tokens, drop self-contradictions), Prioritise (Tier 1 brand acronyms first, Tier 3 generic concepts last), and Trim (at most twelve matched skills and eight missing must-haves). The pipeline is what makes the Tier-based progressive disclosure on the front-end (Section 3.6) feasible.

3.5 Cache hierarchy

The scoring pipeline is expensive: each cold (CV, vacancy) pair requires several model calls. Caching is therefore not an optimisation but a structural requirement. Vakansio is organised around a four-layer back-end cache hierarchy, supplemented by a client-side cache on the front-end. Each layer answers a different question about cost and freshness, summarised in Table 4.

Layer	Storage	TTL	Invalidation
L1 — Response cache	In-process IMemoryCache	5 minutes	TTL expiry
L2 — Scoring cache	PostgreSQL ScoringCache table	None	Version-string bump in the composite key
L3 — Vacancy normalisation	VacancyAnalysisJson column on JobVacancy	Persistent on	Re-normalisation pass
L4 — Scrape cache	In-process IMemoryCache	30 minutes	TTL expiry
Front-end cache	TanStack Query persisted to In-dexedDB	72-hour max age, 10-minute stale time	Cache-buster version bump and multi-tab sync

Table 4: The five-layer cache hierarchy in Vakansio.

L1 — Response cache. The outermost back-end cache stores a fully rendered API response for five minutes, keyed by user identifier, the version of the candidate’s CV, and the search parameters. The result is that repeated identical searches by the same user — for example, refreshing the result page or navigating between tabs in the same session — are served from memory without rerunning the pipeline.

L2 — Scoring cache. The scoring cache is a PostgreSQL table indexed by the composite key (**CvHash**, **VacancyId**, **ScoringVersion**). The CV hash is computed as the SHA-256 of a canonical CV projection, so a non-substantive edit to the CV (whitespace or field-order changes) does not change the key. Each row holds the Composite Judge payload and the six bilingual reason text fields. There is no time-based expiry: invalidation is by version-string bump, as described in Section 3.3. When the scoring pipeline’s composite version

changes, the cache key changes, and every previously stored row becomes unreachable; the row is recomputed lazily on the next request that targets it.

L3 — Vacancy normalisation. The structured representation produced from each vacancy description by the vacancy-normalisation Gemini call is stored as a **jsonb** column on the **JobVacancy** row. This is the most expensive single normalisation step in the pipeline. Once it has been computed once it remains valid until the vacancy is re-extracted, either by the background worker or by a synchronous normalisation fan-out triggered by a subsequent request. The persistence of this layer is what allows the warm-path cost of a typical search to drop substantially relative to the cold-path cost; the measured magnitude is reported in Section 5.

L4 — Scrape cache. The aggregator service that calls into the external sources caches the resolved vacancy list for thirty minutes, keyed by lowercased keywords and location. This is the cache that allows a second identical search to reuse the previous scrape rather than pull every source from the network again.

Front-end cache. On top of the back-end hierarchy, the React front-end caches API responses in a TanStack Query store persisted to IndexedDB. The persisted cache has a maximum age of seventy-two hours and a stale time of ten minutes; writes are throttled to one per second; and a cache-buster version string ensures that incompatible deployments do not serve stale data to users. A multi-tab synchronisation channel keeps several open browser tabs consistent with one another.

Versioning as the shared invalidation rule. The four back-end layers use different keying strategies but share a single invalidation philosophy. Where a time-based expiry is appropriate (L1 response and L4 scrape) it is short, because the underlying data is volatile. Where a time-based expiry is inappropriate (L2 scoring and L3 normalisation) the key itself encodes the version of the procedure that produced the cached value, so a procedure change drops the entire affected slice without any explicit operation. This pattern is what makes the scoring pipeline safe to evolve without coordinated cache flushes.

3.6 Frontend architecture and UX design

The front-end is a single-page React application written in TypeScript and bundled by Vite. It serves five distinct pages — Login, Register, Profile, Job Feed, and Tracker — and acts as the visible surface of the system. This section describes the organisational and user-experience design decisions on the front-end side.

Component organisation. The source tree separates three concerns. The folder **components/ui/** holds design-system primitives — Badge, Button, Card, Input, Modal, Icon — that carry no business-domain knowledge. The folder **components/jobs/** holds feature-level components specific to vacancy display — VerdictBadge, EvidenceChips, VacancyDetailDrawer, and the JobCard variants. The folder **pages/** holds the route-level components that the React Router maps to URLs. Cross-component contracts that would otherwise drift between files are pinned to single-source-of-truth maps: for example, **verdictMeta.ts** defines the verdict label and colour once, and every place that renders a verdict reads from this map rather than maintaining its own table.

State management. Two separate state systems coexist on the front-end, each with a clearly defined role. Zustand stores cross-cutting user-interface and session state (the search term, the active filter selection, the authentication token); TanStack Query stores server data (the result of an API call together with its loading and error state). Strictly local state — for example, the identifier of the currently open detail drawer — remains in React component-level state because it has no readers outside its owning component. The split keeps the React component tree free of imperative state-management code and avoids the common anti-pattern of caching server data in component-level local state.

Job-feed and detail-drawer pattern. The core screen of the application is the Job Feed. Vacancies are displayed as cards in a scrollable list; clicking a card opens a side drawer containing the full vacancy description and the structured explanation. The drawer is a sibling component of the list, not a separate route. This is a deliberate choice: a route change would invalidate the in-page list state (scroll position, filter selections, expanded chip groups) and force a re-render of the result list. The drawer pattern keeps the underlying list mounted and reuses the already-fetched data for the detail view.

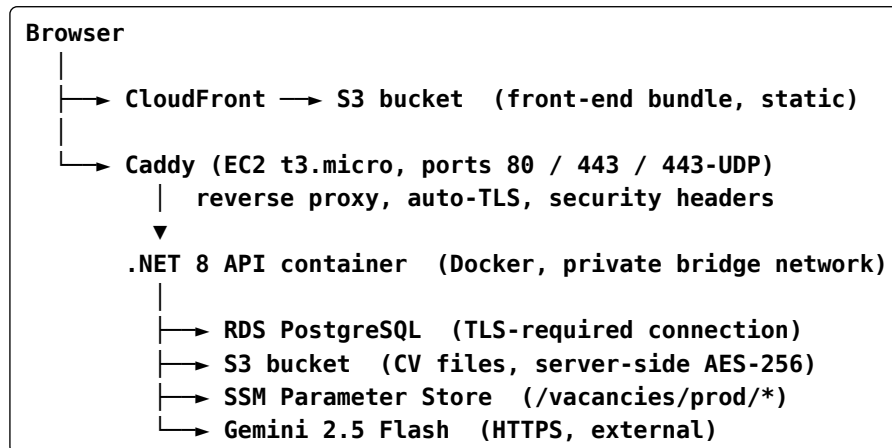
Tier-based progressive disclosure of evidence. Each vacancy carries up to twelve matched-skill chips and eight missing-must-have chips. To prevent the user from being overwhelmed by a wall of chips, and to make scanning useful when the chip count is high, the evidence chips are classified by `evidenceTier.ts` into three tiers based on structural shape. Tier 1 contains brand names and tool acronyms (Figma, Mixpanel, GA4), shown by default. Tier 2 contains hyphenated or multi-word skills, shown with a slight visual hierarchy. Tier 3 contains generic concepts (“communication”, “user-centric approach”), hidden behind a single “+N similar” expansion link. The classifier carries exception lists for known false positives — `LOWERCASE_BRAND_TOOLS` rescues lowercase brand names such as “react” or “spring” from being demoted, and `IMPLICIT_PM_METRICS` rescues product-manager metric phrases — so that the classifier behaves correctly on shapes that the heuristic alone would misclassify. The tier policy mirrors the back-end evidence-pipeline priorities described in Section 3.4.

Verdict badges and bilingual user experience. Each vacancy is labelled with a coloured verdict badge — Strong (green), Partial (blue), Weak (amber), or Mismatch (red) — read from `verdictMeta.ts`. The reason text is bilingual: the back-end returns both English and Ukrainian strings, and the front-end renders whichever language the user has selected through the `LanguageContext`. The internationalisation layer uses paired locale tables (`en.ts` and `uk.ts`), a `useT` hook for translated strings, and a `usePlural` hook for plural-form handling, so no user-visible string is hard-coded.

Persistence subsystem. A dedicated module under `frontend/src/persistence/` owns the lifecycle of the front-end cache. It provides a migration step that upgrades the schema between application versions, a multi-tab synchronisation channel that propagates writes across browser tabs, a quota check that prevents the IndexedDB cache from exceeding browser-imposed limits, and an eviction policy that drops the persisted cache blob when it exceeds a fifteen-megabyte hard cap. The subsystem is what makes the TanStack Query persisted cache (described in Section 3.5) safe to leave running across multiple sessions.

3.7 Deployment topology

The deployment topology of Vakansio reflects three goals: keep the public surface small and standard, lean on managed services wherever the operational gain outweighs the configuration cost, and stay within a single-author project’s operational budget. The result is a single-instance back-end, three managed Amazon Web Services components, and an edge cache for the front-end. The overall shape is shown in Listing 1.



Listing 1: Vakansio production topology on Amazon Web Services.

Compute host. The application back-end runs on a single Amazon Elastic Compute Cloud t3.micro instance in the eu-central-1 region. The instance hosts two Docker containers managed by a single Compose file: Caddy at the public edge and the .NET 8 API container behind it on a private bridge network. There is no separate orchestrator — no Kubernetes, no Elastic Container Service. For a single-author project with one production environment, the operational complexity of an orchestrator outweighs the benefit.

Reverse proxy and TLS. Caddy serves as the public entry point on ports 80, 443, and 443 UDP (the last for HTTP/3). It terminates TLS using automatically issued and renewed Let’s Encrypt certificates, proxies application traffic to the API container, and applies a layer of security headers (HSTS, X-Content-Type-Options, X-Frame-Options DENY, Referrer-Policy). The Caddy configuration sets a 600-second back-end timeout to accommodate cold Gemini calls in the scoring pipeline, which can exceed the default proxy budget on the first request after a cache miss.

Database. PostgreSQL runs on the Amazon Relational Database Service in the same region as the application. The connection is enforced over TLS — the API refuses to start in production unless the connection string carries **SslMode=Require**, **VerifyCA**, or **VerifyFull**. The database is reachable only through a security-group rule that admits the application instance.

Object storage. Uploaded CV files are stored in an Amazon Simple Storage Service bucket with server-side encryption (AES-256), under a per-user key prefix of the form **users/{id}/cv/{timestamp}-{filename}**. The API exposes CV files to the front-end through pre-signed URLs valid for five minutes, rather than streaming the bytes through the API itself.

Secret management. Production configuration secrets — the database connection string, the Gemini API key, the JSON Web Token signing key, and others — live in the AWS Systems Manager Parameter Store under the prefix `/vacancies/prod/*`. The API loads them at startup through the AWS SDK's `AddSystemsManager` extension method and refreshes them in the background every five minutes, so secret rotation does not require a container restart.

Front-end delivery. The compiled front-end bundle is served from a separate Simple Storage Service bucket behind CloudFront. The CloudFront distribution provides geographic edge caching for users across Ukraine, which keeps the time-to-first-byte small even though the application back-end is hosted in Frankfurt.

Identity and access control. The Elastic Compute Cloud instance carries an Identity and Access Management instance profile that grants the application read access to its parameter-store namespace and read-write access to the CV bucket. There are no long-lived AWS access keys in the source tree or in the container's environment.

Single-instance trade-offs. Running one back-end instance has obvious limits: a `t3.micro` is the smallest viable size, and there is no horizontal redundancy. The choice is deliberate. It keeps the operational cost within the AWS twelve-month free tier, eliminates a class of distributed-systems concerns (session affinity, distributed cache coherence) that would consume engineering time without producing thesis-relevant evidence, and is consistent with the scope of a capstone project. The migration path to a multi-instance topology would replace the `Compose` file with an Elastic Container Service task definition and place an Application Load Balancer in front of the instances; the layer separation described in Section 3.3 makes this migration largely mechanical.

3.8 Observability and security design

The fourth and fifth architectural drivers from Section 3.1 — per-stage evaluability with cost transparency, and security with data protection — share a common pattern. Both rely on cross-cutting infrastructure that has to be present at every stage of the pipeline without coupling the layers. This section describes the design responses for each.

Per-stage cost tracking. Every Gemini call in the pipeline goes through an `AsyncLocal`-backed accumulator declared in `Application/Common/Diagnostics/CostBreakdown.cs`. The accumulator is request-scoped, so parallel scoring tasks within the same incoming HTTP request share a single record while concurrent requests do not interfere with one another. For each stage it tracks the number of calls, the elapsed wall-clock time, the input and output token counts, and the dollar cost computed from the current Gemini 2.5 Flash unit prices. The choice to host the accumulator in the Application layer rather than in Infrastructure is deliberate: it allows both the orchestrating Application handler and the concrete Infrastructure services to write into the same accumulator without the Application layer reaching downward into Infrastructure, and it preserves the Clean Architecture dependency direction.

Persistent cost ledger. At the end of each request the accumulator is flushed to a PostgreSQL table named `gemini_cost_log`, modelled by the `GeminiCostLogEntry` entity. The accumulator emits one entry per stage that was touched during the request, and each

entry is persisted as a row with columns for the user identifier, the request identifier, the request kind (for example `v6_search`, `cv_normalize`, or `worker_backfill`), the stage name (`judge_batched`, `reason_batched`, `skill_expansion`, and so on), the number of calls, the wall-clock duration, the input and output token counts, the dollar cost, and the keyword string when applicable. An administrator can query the table directly through `/api/admin/cost`, which returns the data as comma-separated values grouped by an optional dimension. The numbers themselves are reported in Section 5.

Best-effort writes. A failure in the cost-logging path must never break a user response. The implementation in `Infrastructure/Services/CostLogService.cs` catches every database exception and continues, on the principle that a missing log entry is preferable to a failed search. Structured logging through `ILogger` at every stage means the loss is visible in the application logs and can be diagnosed after the fact.

Health endpoints. The API exposes two health endpoints. The lightweight `/health` endpoint returns a fixed liveness response without touching any dependency; it is consumed by Caddy and by the Docker container's healthcheck on each interval. The heavier `/health/ready` endpoint performs a database round-trip through `IDatabaseHealthService` and is used during deployment to confirm that the new container is ready to serve traffic before old containers are stopped.

Authentication. The API uses stateless JSON Web Tokens for authentication, configured through the `AddJwtBearer` extension in `Program.cs`. Passwords are hashed with the BCrypt.Net-Next library, which exposes the standard work-factor parameter and is regularly audited. There is no in-process session state, which keeps the application straightforward to scale horizontally if a future deployment needs it.

Rate limiting at the application layer. Request rate limiting is implemented as ASP.NET Core middleware inside the API project rather than in Caddy. The Caddy layer is deliberately kept narrow — TLS termination, security headers, and request forwarding — so that the application has a single source of truth for the policy and can apply per-route limits informed by application context.

Secret storage. Production secrets live in the AWS Systems Manager Parameter Store, as described in Section 3.7. No production secret enters the source tree.

Startup guards. Two production-only startup guards turn configuration mistakes into immediate, loud failures. The first refuses to start the application unless the database connection string carries one of `SslMode=Require`, `SslMode=VerifyCA`, or `SslMode=VerifyFull`. The second refuses to start unless the `Cors:AllowedOrigins` list is non-empty, which prevents a configuration mistake from leaving the application open to cross-origin calls from any origin. Both guards throw at startup and emit a clear log line; a misconfigured deployment cannot drift into a quietly insecure state.

GDPR cascade delete. User-account deletion is implemented as a transactional cascade in `Application/User/Commands/DeleteCurrentUser/DeleteCurrentUserHandler.cs`. The handler reads the CV file key from the user record, runs a transactional cascade through the database (the user's tracker applications, the relevance-explanation rows keyed on the user's current CV version identifier, and the user-profile row itself), and finally issues a best-effort S3 deletion of the CV file. The database is treated as the

authoritative artefact for the right-to-be-forgotten obligation; an orphan blob in the S3 bucket is recoverable via the bucket's lifecycle policy and does not violate the deletion semantics.

Soft-determinism on model calls. Every Gemini call carries a short per-call timeout — eight seconds in the inner scoring-loop reason call, fifteen to twenty-five seconds in the dedicated Composite Judge, batched Judge, and batched reason services — and falls back to a deterministic template if the timeout fires or the response fails schema validation. A misbehaving language-model call therefore cannot block the request loop or produce a non-renderable output.

4 | Implementation

This chapter walks through how the Vakansio design from Section 3 was turned into a working, production-deployed system. The first section documents the development methodology and the conventions that guided the code base. The next three sections walk layer by layer through the Domain, Application, and Infrastructure projects. A fifth section describes the front-end implementation. A sixth section covers deployment, operations, and operating costs. A final section presents a small number of engineering narratives — challenges encountered during development that are worth reporting in detail because they illustrate how the architecture absorbed the change.

4.1 Development methodology

This section describes the development process behind Vakansio: how the work was organised, what conventions guided the code, what quality measures were applied, and which tools were used.

Iterative versioned development. The project was developed by a single author over several months. Rather than fixed-length sprints, the work was organised around explicit versioned releases of the scoring pipeline. Each version corresponded to an observable change in system behaviour: a rebalance of the scoring weights, the introduction of the global-vocabulary path in version 6.3 that replaced the earlier per-entity skill expander, and several contract-drift fixes documented later in Section 4.7. The version string is also the cache-invalidation key described in Section 3.3, so each iteration is both an engineering increment and a deployable artefact with a defined effect on the persisted caches.

Prototype-first approach. The system grew outward from a minimal core. The first working version contained only the deterministic seven-axis sub-scorer and a thin API. The Composite Judge stage was added next, then the batched reason generator, then the front-end tier-based evidence disclosure, then the per-stage cost-tracking subsystem. Each step was validated against real CV–vacancy pairs on the deployed instance before the next layer was added. This approach kept the integration cost of any single change small enough that the system could be returned to a working state quickly when a change misbehaved.

Coding standards and conventions. The back-end is written in C# on .NET 8 with nullable reference types enabled across every project. The code base follows the default C# naming conventions — **PascalCase** for public members, **camelCase** for local variables and parameters, **IPascalCase** for interfaces. Each Command–Query Responsibility Segregation handler is a single class with exactly one public **Handle** method. The Clean Architecture project references are enforced at the build level: the Domain project declares zero package references in **Domain/Domain.csproj**, and the Application project references only Domain. On the front-end, TypeScript is built with **noUnusedLocals**, **noUnusedParameters**, and **noFallthroughCasesInSwitch** enabled, so the build fails on dead code and missed switch cases; ESLint enforces the recommended TypeScript and React-Hooks rule sets. Cross-component contracts that would otherwise drift between files are pinned to single-source-of-truth maps such as **verdictMeta.ts** and **evidenceTier.ts**.

Quality assurance approach. Three quality measures run in parallel. First, unit tests cover the pure Domain rules and the sub-score calculators, where the input–output mapping is deterministic; the test suite is detailed in Section 5. Second, every observable version of the system was acceptance-tested by the author against the deployed instance at dsus1dizgh006.cloudfront.net on a fixed set of CVs and search queries before the version was considered shipped. Third, an offline evaluation harness — separate from the production code path — computes ranking-quality and reason-quality metrics on an LLM-judged gold set; the methodology and results are reported in Section 5. The continuous cost-tracking subsystem described in Section 3.8 complements the test suite by surfacing performance regressions as cost changes in the `gemini_cost_log` table.

Tooling and workflow. JetBrains Rider served as the primary IDE for the back-end. PostgreSQL was used during development and through Amazon’s Relational Database Service in production. Docker and Docker Compose containerised the back-end for both local runs and the production deployment process described in Section 4.6. A continuous-integration and continuous-deployment pipeline was not implemented within the prototype scope and is recorded as future work in Section 6.

4.2 Domain layer walkthrough

This section walks through the Domain project — the innermost layer of the Clean Architecture decomposition introduced in Section 3.3. The project lives under `Domain/` and contains every artefact that describes the business of candidate–vacancy matching without reference to any external technology. The file `Domain/Domain.csproj` declares zero package references and no project references, so the layer can be reasoned about and unit-tested in isolation from Entity Framework Core, the Gemini software development kit, the Amazon Web Services adapters, and every other infrastructure concern.

Entities. The project defines seven entities under `Domain/Entities/`. `JobVacancy` models a vacancy as it has been extracted from a source, together with its normalised analysis as a `jsonb`-stored payload (the L3 cache layer in Section 3.5). `UserProfile` holds the user record, the pointer to the current CV version in object storage, and the cached normalisation of the CV. `ApplicationTracker` models the per-user record of a job the user has decided to act on. `ScoringCacheEntry` is the row backing the L2 scoring cache, keyed by the composite (CV hash, vacancy identifier, scoring version) triple. `SkillVocabularyEntry` backs the global-vocabulary path introduced in version 6.3. `GeminiCostLogEntry` is the per-call row in the cost ledger described in Section 3.8. `SavedUrl` records vacancies the user has manually added by URL.

Value objects and enumerations. Two value objects live under `Domain/ValueObjects/`. `RelevanceScore` wraps a floating-point score together with the pipeline stage that produced it; the constructor rejects values outside the closed interval $[0, 100]$, and `ToPercent()` renders the score for display. `Salary` wraps a minimum amount, a maximum amount, a currency, and a raw-text fallback for cases where the source did not expose a structured value. Five enumerations live under `Domain/Enums/`: `ApplicationStatus` for the tracker workflow, `JobSource` with seven members covering the six aggregators (Djinni, WorkUa, LinkedIn, Jooble, RobotaUa, DOU) and a `Manual` fallback for user-

submitted URLs, **ScoringStage** for the per-stage cost annotations, **SeniorityLevel**, and **WorkFormat**.

Pure scoring rules. The folder **Domain/Scoring/** is the substantive core of the Domain layer. It contains the deterministic rules that operate on already-normalised inputs and produce typed outputs. **CvHasher** computes the SHA-256 hash of a canonical projection of the CV-summary JSON — the sorted skill lists, the target roles, the seniority, the English level, and the total experience in months — so a non-substantive edit to the CV does not change the L2 cache key. **GenericGapFilter** drops generic terms (“communication”, “teamwork”) from the missing-skill list before the result is shown to the user, on the grounds that those are not actionable gaps. **LanguageGapDetector** recognises the case where the vacancy’s required English level strictly exceeds the candidate’s, using a Common European Framework of Reference ranking from A1 to C2. **RoleFamily** and **RoleFamilyDetector** provide the deterministic family routing described in Section 3.4. **SubScores** is a **record** that carries the seven per-axis scores; the current weights live in **ScoringServiceV2.cs:138–147** rather than in the record itself, so a rebalance changes only one file. **ScoringResult** aggregates the headline score, the sub-scores, the verdict, and the evidence chips into the single typed output of the scoring pipeline.

Verdict labels and the bilingual reason path. The file **Domain/Scoring/Verdict.cs** ties together two cross-cutting concerns: the categorical verdict label and the bilingual output. The enumeration carries four members — **Mismatch**, **WeakMatch**, **PartialMatch**, and **StrongMatch**. The companion **VerdictExtensions** class exposes **FromScore(double)**, which maps a composite score to a verdict at the cuts 0.25, 0.50, and 0.75; these are user-interface display cuts and are deliberately distinct from the Composite Judge calibration anchors documented in Section 3.4. The extension class also exposes **ToEnglishText()**, **ToUkrainianText()**, and **ToShortName()** the first two carry the user-visible bilingual labels (for example, “Strong match” and “Сильна відповідність”), while the third carries the short label that the front-end **verdictMeta.ts** map keys on. Keeping all three vocabularies in the Domain layer means that any change to the verdict-label contract is a single-file edit, and that no Infrastructure or Application class can quietly disagree about what a “Strong match” is.

Interfaces declared in the Domain layer. Two folders carry interface declarations. **Domain/Interfaces/Repositories/** contains seven repository contracts — **IJobVacancyRepository**, **IUserProfileRepository**, **IApplicationRepository**, **ISavedUrlRepository**, **IScoringCacheRepository**, **ISkillVocabularyRepository**, and **IGeminiCostLogRepository**. Repository contracts live in Domain rather than in Application because the aggregate they serve is itself a Domain concept; the layer that owns the entity should own the abstraction through which the entity is read and written. **Domain/Interfaces/Services/** contains three additional contracts that describe domain operations whose implementation is technology-specific but whose intent is not: **IDeduplicationService** for cross-source vacancy deduplication, **IJobSourceService** for the per-source adapter contract, and **IRelevancePipeline** for the pipeline orchestration described in Section 3.4. Other contracts that are application-level rather than domain-level — for example **ICvFileStorage**, **ICostLogService**, **IScoringService**,

IBatchedJudgeService, and **IBatchedReasonService** — live under **Application/Common/Interfaces/** instead, and are covered in Section 4.3.

4.3 Application layer walkthrough

This section walks through the Application project — the use-case layer of the Clean Architecture decomposition. The file **Application/Application.csproj** declares exactly one project reference, to Domain, and five package references: **FluentValidation** 12.0.0, **FluentValidation.DependencyInjectionExtensions** 12.0.0, **MediatR** 14.0.0, **Microsoft.Extensions.Caching.Memory** 8.0.1, and **Microsoft.Extensions.DependencyInjection.Abstractions** 10.0.0. There is no reference to Entity Framework Core, no reference to the Gemini software development kit, and no reference to any AWS SDK — every infrastructure-bound dependency lives behind an interface declared either in Domain or in this layer.

Feature-folder CQRS organisation. The use cases are organised under four feature folders — **Jobs/**, **Tracker/**, **User/**, and **Eval/** — each split into a **Commands/** subfolder for state-changing operations and a **Queries/** subfolder for read-only operations. Each use case lives in its own subfolder named after the operation. A typical use case is composed of three files: a ***Command.cs** or ***Query.cs** that declares the request DTO and implements **IRequest<TResponse>** a ***Handler.cs** that implements **IRequestHandler<TRequest, TResponse>** with one **Handle** method; and, where input validation is required, a ***Validator.cs** that derives from **AbstractValidator<TRequest>**. Representative examples include **Jobs/Commands/AddManualJobUrl/**, **Jobs/Queries/GetAggregatedJobsV6/**, **Tracker/Commands/AddToTracker/**, and **User/Commands/DeleteCurrentUser/**. The folder layout is what makes the CQRS split visible at a glance — a reader can answer “what does the system do?” by scanning the folder names alone.

The orchestrator handler. The largest handler in the project is **Jobs/Queries/GetAggregatedJobsV6/GetAggregatedJobsV6Handler.cs**, at roughly one thousand lines of code. It is the entry point for the search-and-score pipeline whose stages are documented in Section 3.4. Despite its size, the handler contains no direct Gemini call, no Entity Framework Core context, and no scraping logic. Every collaborator — the aggregator service, the scoring service, the Composite Judge, the batched reason generator, the skill-vocabulary service, the cost logger — is injected through its interface declared under **Application/Common/Interfaces/** or under **Domain/Interfaces/**. The handler’s job is to compose those collaborators in the right order, to manage the per-request cost-tracking scope, and to assemble the response DTO; the actual work of calling out to the Gemini API or the database lives in Infrastructure.

Options pattern for tuning knobs. Pipeline tuning knobs are bound through the **IOptions<T>** pattern rather than left as inline constants. The first such POCO is **Application/Common/Configuration/ScoringOptions.cs**, which exposes a single property **SyncNormalizeTimeoutSeconds** (default 300). The value is read from the “**Scoring**” section of **appsettings.json** at startup, with the property default as the fallback when the section is absent. The XML documentation on the class records the intent that further magic numbers in the codebase — the scrape cache TTL, the response cache TTL, and the batched-judge and batched-reason chunk sizes — should migrate to this file or to

sibling ***Options** POCOs over time, so that configuration is a first-class concern rather than scattered throughout the implementation.

Common interfaces and DTOs. The folder **Application/Common/Interfaces/** declares the abstractions through which the use cases reach out to the rest of the system. The core ones include **ICvFileStorage** for object-storage access, **ICostLogService** for the persistent cost ledger, **IScoringService**, **ICompositeJudgeService**, **IBatchedJudgeService**, and **IBatchedReasonService** for the LLM-bound stages of the scoring pipeline, **ISkillVocabularyService** for the global-vocabulary path, **ICvDomainRouter** and **IVacancyDomainRouter** for the domain routing described in Section 3.4, and **ICurrentUserService** for the request-scoped user identity. The folder **Application/DTOs/** holds the carrier objects that cross the layer boundary into the API project, including **JobVacancyV6Dto**, **RankedJobDto**, **ApplicationTrackerDto**, and **UserProfileDto**. DTOs are independent of the Domain entities they shadow, so a change to a stored entity does not automatically reshape the public API surface.

Cross-cutting MediatR pipeline behaviour. Input validation runs as a MediatR pipeline behaviour rather than as a call inside each handler. The class **Application/Common/Behaviors/ValidationBehavior.cs** implements **IPipelineBehavior<TRequest, TResponse>** on every dispatched request, it collects every registered **IValidator<TRequest>** from the dependency-injection container, runs them, and throws a **FluentValidation.ValidationException** with the accumulated failures if any rule fails. Only if the validators all pass does control reach the actual handler. The wiring lives in **Application/DependencyInjection.cs**, whose **AddApplication** extension method registers all MediatR handlers from the current assembly, registers all **AbstractValidator** subclasses from the current assembly, and registers the pipeline behaviour itself as a transient open-generic service.

Per-stage cost accumulator. The per-stage cost-tracking subsystem introduced in Section 3.8 is implemented in **Application/Common/Diagnostics/CostBreakdown.cs** as a static class backed by an **AsyncLocal<Accumulator?>** field. The orchestrator handler opens a scope at the start of the request through **CostBreakdown.BeginScope()** each Gemini-calling service in the Infrastructure layer reports its stage statistics through **CostBreakdown.Track(stage, ms, inputTokens, outputTokens)** and at the end of the request the handler reads the snapshot through **GetSnapshot()** and persists the rows. The **AsyncLocal** storage is what makes the accumulator both request-scoped — two concurrent HTTP requests do not interfere with one another — and parallel-safe within a single request, so the up-to-three parallel reason-generation chunks described in Section 3.4 all write into the same accumulator. The pricing constants for the Gemini 2.5 Flash short-context tier (input \$0.30 and output \$2.50 per million tokens at the time of writing) are private constants in this file; a price change is a one-line edit. The file's class-level comment makes the architectural choice explicit: the accumulator lives in Application rather than Infrastructure so that both the orchestrating handler and the concrete Infrastructure services can write to and read from it without breaking the Clean Architecture dependency direction.

4.4 Infrastructure layer walkthrough

This section walks through the Infrastructure project — the layer that translates the abstractions declared in Domain and Application into concrete calls against the database, the Gemini API, the AWS services, and the seven external job sources. The file **Infrastructure/Infrastructure.csproj** declares one project reference, to Application, and a focused set of package references. Persistence is provided by Entity Framework Core 8.0.10 together with the Npgsql and Pgvector providers for PostgreSQL. AWSSDK.S3 4.0.15 backs the object-storage adapter, BCrypt.Net-Next 4.0.3 backs password hashing, HtmlAgilityPack 1.12.0 backs the HTML scrapers, System.ServiceModel.Syndication 8.0.0 backs the RSS feed parser, and PdfPig 0.1.14 backs the CV-PDF extractor. The Microsoft.Extensions.Hosting and Http packages cover the background workers and the typed HTTP-client wiring. Every collaborator the application uses against the outside world is contained inside this layer.

Persistence with Entity Framework Core. The folder **Infrastructure/Persistence/** holds the database integration. **AppDbContext** is the single **DbContext**, with one **DbSet** per persisted entity. The companion **AppDbContextFactory** provides the design-time factory that the EF Core command-line tools use during migrations. The folder **Persistence/Configurations/** contains six **IEntityTypeConfiguration<T>** classes, one per Domain entity that needs explicit column shaping. The folder **Persistence/Migrations/** contains ten sequential migrations from **20260314_InitialCreate** through **20260528_AddSkillExpansionColumns**. The folder **Persistence/Repositories/** contains seven repository implementations, one for each Domain repository contract. **Persistence/Entities/RelevanceExplanation.cs** is the only entity declared in Infrastructure rather than in Domain; it carries no invariants and exists purely as a denormalised projection of LLM-generated explanations. The cascade-delete behaviour described in Section 3.8 is implemented through Entity Framework Core’s **OnDelete(DeleteBehavior.Cascade)** annotations declared in **UserProfileConfiguration** and reinforced by the **DeleteCurrentUserHandler** in Application.

Job-source adapters and aggregation. The folder **Infrastructure/JobSources/** contains the seven adapters that pull vacancies from the external sources, split by access pattern. Two are REST API clients: **JoobleApiService** and **RobotaUaApiService**, both under **JobSources/Api/**. One is an RSS feed parser: **DouRssFeedService** under **JobSources/Rss/**, built on **System.ServiceModel.Syndication**. Four are HTML scrapers under **JobSources/Scraping/**: **DjinniScraperService**, **LinkedInGuestService**, **WorkUaScraperService**, and **ManualUrlScraperService**, all driven by **HtmlAgilityPack**. Shared helpers — **JobDescriptionFetcher** for per-vacancy detail fetches, **ScrapedToDomainMapper** for the cross-adapter mapping into **JobVacancy**, **ScrapedVacancyDto**, and **UkrainianDateParser** for the date-format normalisation — sit next to the adapters. Every adapter implements the same **IJobSourceService** contract declared in Domain. **JobAggregation/JobAggregationService** orchestrates the request. It resolves every registered **IJobSourceService**, fans the call out in parallel, and catches per-source exceptions so that a single failing source does not break the response. The com-

bined result then passes through **Deduplication/DeduplicationService** and is cached in **IMemoryCache** with **AbsoluteExpirationRelativeToNow = TimeSpan.FromMinutes(30)** under the key **scrape:{lowercased keywords}:{lowercased location}**. The thirty-minute window is a static readonly field declared at **JobAggregationService.cs:37**.

Scoring pipeline implementation. The folder **Infrastructure/RelevancePipeline/V2/Scoring/** implements the pipeline whose stages were designed in Section 3.4. **ScoringServiceV2** is the implementation of **IScoringService** and the orchestrator of the deterministic-plus-LLM scoring flow; the weight table at lines 138–147 is the canonical source of the seven sub-score weights. The folder **SubScoreCalculators/** contains the seven per-axis calculators — **SkillMatchCalculator**, **SeniorityMatchCalculator**, **ExperienceMatchCalculator**, **LanguageMatchCalculator**, **EducationMatchCalculator**, **RoleIntentMatchCalculator**, and **DomainAlignmentCalculator** — all stateless and registered as singletons. The deterministic safety layer consists of **AntiFlagEvaluator** and **ScoringCapService** the latter applies the floor of 0.20, the ceiling of 0.88, and the asymmetric caps described in Section 3.4. The LLM-bound services are **GeminiCompositeJudgeService** (the single-pair Judge), **GeminiBatchedJudgeService** (the chunked Judge that scores the pre-filtered top forty in roughly six to eight calls), and **GeminiBatchedReasonService** (the batched bilingual reason generator). **JudgePromptCore** holds the prompt body and the version string that participates in the cache key. Skill expansion takes two forms. The current global-vocabulary path uses **Application/Common/SkillVocabulary/SkillVocabularyService**, exposed through **ISkillVocabularyService** and backed by **GeminiBatchSkillExpander**. The earlier per-entity expander **GeminiSkillExpansionService** remains registered as a fallback for the legacy code path but is no longer the primary entry point.

Module-based CV and vacancy normalisation. The folder **RelevancePipeline/V2/CvNormalization/** and the folder **RelevancePipeline/V2/VacancyNormalization/** implement the structured-extraction stages that convert free-text CVs and vacancy descriptions into the typed JSON payloads consumed by the scoring pipeline. Both follow the same five-component shape. A deterministic keyword router (**KeywordCvDomainRouter** or **KeywordVacancyDomainRouter**) implements the corresponding Application contract. A ***ModuleResolver** then resolves the routed domain to a concrete normalisation module. The per-domain modules carry the slot content for their domain. A ***PromptBuilder** composes the slots into the final Gemini prompt, and a ***PostProcessor** applies deterministic clean-up to the model output. The CV side ships two modules — **TechCvNormalizationModule** and **GenericCvNormalizationModule**. The vacancy side ships six — **TechVacancyNormalizationModule**, **HrVacancyNormalizationModule**, **HealthcareVacancyNormalizationModule**, **MarketingVacancyNormalizationModule**, **SalesVacancyNormalizationModule**, and **GenericVacancyNormalizationModule** as the catch-all fallback. Adding a new domain is a four-step procedure documented in the composition root’s comments: extend the domain enum, implement the module interface, register the module in **DependencyInjection.cs**, and extend the keyword router. No other code change is required.

AWS adapters, ML API adapters, and operational services. The folder **Infrastructure/Services/** holds the small adapters that bridge into the rest of the

platform. **S3CvFileStorage** implements **ICvFileStorage** with the AWS SDK and AES-256 server-side encryption, generating pre-signed URLs with five-minute lifetimes under the key prefix **users/{userId:N}/cv/{timestamp}-{filename}**. It is registered conditionally: when the **S3:CvBucket** configuration value is present the real adapter binds; otherwise **NoOpCvFileStorage** binds so the API still boots in development. **CostLogService** implements **ICostLogService** with best-effort writes to the **gemini_cost_log** table — every database exception is caught and logged so that a logging failure cannot break a user response. **DatabaseHealthService** implements **IDatabaseHealthService** and performs the database round-trip used by the **/health/ready** endpoint. The folder **Infrastructure/MlApi/** contains adapters to a separate, optionally deployed Python ML service at the default address **http://localhost:8000**. **MlApiScoringService** provides bi-encoder scoring, **MlApiEmbeddingService** provides multilingual-e5-base embeddings, **MlApiReasoningService** provides Qwen2.5-3B-Instruct reasoning, and **MlApiFactualityService** provides the MiniCheck factuality check used by the reason-quality evaluation reported in Section 5. The ML API path is independent from the primary Gemini path and is wired through typed **AddHttpClient<TInterface, TImpl>** registrations with their own per-service timeouts.

Background workers and the composition root. The folder **Infrastructure/Workers/** contains four **BackgroundService** implementations — **JobEmbeddingWorker**, **CvSummaryWorker**, **VacancyAnalysisWorker**, and **ReasoningWorker**. All four are flag-gated and default off in production. **JobEmbeddingWorker** is gated by **ML:EnableEmbeddingWorker** the two cost-sensitive Gemini-driven workers (**CvSummaryWorker**, **VacancyAnalysisWorker**) are gated by **BackgroundWorkers:EnableCvSummary** and **BackgroundWorkers:EnableVacancyAnalysis** respectively; **ReasoningWorker** is intentionally not registered at all because the inline batched reason generator superseded its role. The hot path replaces the disabled background normalisation with the synchronous fan-out documented in Section 4.3, bounded by the **SyncNormalizeTimeoutSeconds** budget. The composition root **Infrastructure/DependencyInjection.cs** wires every contract above to its implementation through an explicit registration block; there is no MEF-style assembly scan in Infrastructure, by design. The same file configures **AddMemoryCache(opts => opts.SizeLimit = 1000)** as the shared in-memory cache backing both the thirty-minute scrape cache and the five-minute response cache from Section 3.5.

4.5 Frontend implementation details

This section turns the front-end design from Section 3.6 into a concrete file map and documents the implementation choices that the design section did not cover. The front-end stack is React 19.2.5 with TypeScript 6.0.2 and the Vite 8.0.10 bundler. Routing across the five pages — Login, Register, Profile, Job Feed, and Tracker — is handled by react-router-dom 7.14.2. The production build runs as **tsc -b && vite build**, so a TypeScript compilation error fails the build before Vite ever produces a bundle. Linting runs as **eslint** . under the configuration described in Section 4.1.

Source-tree organisation. The folder `src/components/ui/` holds six design-system primitives, and the folder `src/components/jobs/` holds twelve feature-level components, including the single-source-of-truth maps `verdictMeta.ts` and `evidenceTier.ts` introduced in Section 3.6. The folder `src/pages/` contains one subfolder per route. The folder `src/api/` contains six files: the shared axios client and five per-feature wrappers (`authApi.ts`, `jobsApi.ts`, `profileApi.ts`, `trackerApi.ts`, `userApi.ts`). The folder `src/store/` holds the two Zustand stores. The folder `src/i18n/` holds the internationalisation layer. The folder `src/persistence/` holds the five-file persisted-cache subsystem. The remaining folders are `src/hooks/`, `src/styles/`, `src/types/`, and `src/assets/`.

API client and JWT handling. The file `src/api/client.ts` declares a single `axios.create()` instance shared by every per-feature wrapper. The base URL is read from the `VITE_API_BASE_URL` environment variable, with `http://localhost:5180/api` as the fallback when the variable is unset. The default timeout is five minutes (`300_000` milliseconds), which leaves headroom for a cold v6 search whose first-time pipeline can take up to two minutes. A request interceptor injects the header `Authorization: Bearer ${token}` whenever a JSON Web Token is present in `localStorage`. A response interceptor handles the unauthenticated case: when a `401` response is observed and the user is not already on the login route, the three persisted identity keys (`token`, `userId`, `email`) are removed and the browser is redirected to `/login`. The wrappers in `src/api/` issue typed calls against this client; each backend feature has its own wrapper, so a controller change touches a single file on the front-end.

Vite dev-mode proxy. The file `vite.config.ts` configures `server.proxy` to forward every request under `/api` to `https://api.vakansio.online` with `changeOrigin: true`. The browser only ever issues same-origin requests to `localhost:5173/api/*`. Vite forwards them server-side to the production API. Two practical consequences follow. First, the production `Cors:AllowedOrigins` allow-list (Section 3.8) does not need to include the developer's local host, because no cross-origin request ever leaves the browser. Second, the development iteration loop runs against real production data and the real production scoring caches. A local change can therefore be evaluated against the same vacancies and the same `ScoringCache` rows as the deployed instance. Restoring local back-end development is a single change: set `VITE_API_BASE_URL=http://localhost:5180/api` in `.env.development` and remove the `server.proxy` block.

State management implementation. The split designed in Section 3.6 maps onto two Zustand stores and the TanStack Query layer. `src/store/authStore.ts` holds authentication state — the JSON Web Token, the user identity, and the helpers that read or write the persisted `localStorage` keys. `src/store/jobStore.ts` holds the cross-cutting search state: the current `searchParams` object (`keywords`, `location`, and the `runRelevancePipeline` flag), together with the cached server payload (`jobs`, `totalCount`, `duplicatesRemoved`) and the loading and error flags. Server data — the responses from `/api/jobs/v6` and the related endpoints — also lives in TanStack Query (`@tanstack/react-query 5.100.3`) under its own keyed cache, and the front-end never copies that cache into the Zustand stores. Strictly local state, such as the identifier of the currently open vacancy drawer, remains in component-level `useState` because no other component reads it.

Persistence subsystem implementation. The folder `src/persistence/` contains five files, one per concern of the design described in Section 3.6. `queryPersister.ts` builds an async-storage persister from `@tanstack/query-async-storage-persister` over an IndexedDB store backed by `idb-keyval 6.2.1` `src/main.tsx` then wraps the application in `PersistQueryClientProvider` and passes the persister together with the cache-buster string (`v6.7.2` at the time of writing). Advancing the buster string causes the persist-client library to drop the persisted blob on the next mount. `migration.ts` performs a one-shot migration from the legacy `localStorage`-backed cache blob to the new IndexedDB key on startup and exposes a `clearAllQueryCache()` helper used during logout. `multiTabSync.ts` opens a `BroadcastChannel` so that a write in one tab invalidates the corresponding query in every other open tab. `storageQuota.ts` exposes three diagnostic helpers — an IndexedDB-availability probe (`isStorageAvailable`), a wrapper over `navigator.storage.estimate()` (`getStorageEstimate`), and a `QuotaExceededError` classifier — that the rest of the persistence subsystem uses for diagnostics and for graceful degradation. `cacheEviction.ts` implements the fifteen-megabyte hard cap by dropping the persisted blob in its entirety (`HARD_LIMIT_BYTES = 15 * 1024 * 1024`) once the threshold is crossed, rather than evicting individual entries.

Internationalisation implementation. The folder `src/i18n/` implements the bilingual user experience. `LanguageContext.tsx` provides the React context that propagates the active language down the tree. `useT.ts` exposes a hook that resolves a translation key against the active locale table. `usePlural.ts` exposes a companion hook that handles the plural-form rules for English and Ukrainian, which differ on the cardinality boundaries. `translations.ts` declares the type-safe key catalogue, and the two locale tables (`locales/en.ts` and `locales/uk.ts`) carry the actual strings. No user-visible string is hard-coded in a component file; every label, message, and verdict text passes through `useT` or through the verdict-text helpers in `verdictMeta.ts`.

4.6 Deployment, operations, and operating costs

This section documents how the topology designed in Section 3.7 is built, shipped, and run, and what it costs to keep the system in production. The build artefact is a single Docker image produced from the repository root; the production environment is a single Amazon Elastic Compute Cloud host that runs two containers through Docker Compose; the deployment is a manual but reproducible procedure.

Container build. The file `Dockerfile` is a three-stage build that produces a slim, multi-architecture runtime image (`linux/amd64` and `linux/arm64`). The first stage, `sdk-restore`, copies only the four `.csproj` files into the build context and runs `dotnet restore`. The layer cache therefore only invalidates when the package graph itself changes. The second stage, `sdk-build`, copies the rest of the source and runs `dotnet publish -c Release --no-restore /p:UseAppHost=false` into `/app/publish`. The third stage starts from the official `mcr.microsoft.com/dotnet/aspnet:8.0` image, installs `curl` for the container health probe, creates a non-root `app` user, copies the published output, exposes port 8080, and declares a container-level `HEALTHCHECK` that polls `/health` every thirty seconds.

Compose topology. The file **docker-compose.production.yml** defines two services on a private bridge network **vacancies-net**. The **caddy** service runs the Caddy 2.8-alpine image with ports 80, 443, and 443 UDP (for HTTP/3) exposed to the host, and two persistent volumes carry the Let's Encrypt account material and configuration. The **api** service runs the **vacancies-api:latest** image with port 8080 only exposed inside the network — the host never publishes it. A memory cap of seven hundred megabytes protects the t3.micro host (which has roughly nine hundred megabytes of usable memory) from a runaway .NET allocation. The environment variable **BackgroundWorkers__EnableVacancyAnalysis=false** explicitly disables the vacancy-analysis worker; the compose file's inline comment records the rationale that an always-on background worker would burn three to six dollars per day in Gemini calls even with no user traffic. Caddy's **depends_on: api { condition: service_healthy }** ensures the reverse proxy does not start serving traffic before the API container reports healthy.

Caddy reverse proxy. The **Caddyfile** binds the public hostname from the environment variable **API_HOSTNAME** (the deployed instance uses **api.vakansio.online**) and obtains a Let's Encrypt certificate automatically. Hardening headers are applied uniformly: **Strict-Transport-Security: max-age=31536000; includeSubDomains**, **X-Content-Type-Options: nosniff**, **X-Frame-Options: DENY**, and **Referrer-Policy: strict-origin-when-cross-origin**. The default Caddy **Server** header is suppressed. The two health routes pass through to the API. Every other request is reverse-proxied to **api:8080** with the original client information preserved through **X-Real-IP**, **X-Forwarded-For**, **X-Forwarded-Proto**, and **X-Forwarded-Host**, so that the API's rate limiter partitions by the real client identifier rather than by the proxy's. The proxy timeouts are six hundred seconds for read, write, and response-header phases to accommodate cold v6 Gemini calls that can take sixty to one hundred and thirty seconds end-to-end. Response compression uses **zstd** and **gzip**. Rate limiting and the CORS policy are intentionally not configured in Caddy; both live in the API project, so that the policy has a single source of truth.

Deployment procedure. Deployment is a manual but reproducible script. On the developer workstation, the image is built (**docker build -t vacancies-api:<sha> -f Dockerfile .**), then saved and compressed (**docker save vacancies-api:<sha> | gzip > image.tar.gz**), and copied to the production host with **scp**. On the host, the image is loaded (**docker load < image.tar.gz**), retagged to **latest**, and rolled with **docker compose -f docker-compose.production.yml up -d**. No container registry is involved. The front-end deployment compiles the bundle with **npm run build**, uploads it to the static-hosting bucket with **aws s3 sync dist/ s3://...**, and issues a CloudFront invalidation for the affected paths. The repository has no **.github/workflows/** directory; a continuous-integration and continuous-deployment pipeline is recorded as future work in Section 6.

Health probes and rollout safety. The two-tier health endpoint design from Section 3.8 is realised here. The Docker container's own **HEALTHCHECK** polls **/health** with **curl** every thirty seconds, with a five-second timeout, three retries, and a thirty-second start period. Caddy refuses to start until that probe reports healthy through **depends_on: service_healthy**. The deeper **/health/ready** endpoint (implemented by **DatabaseHealthService** from Section 4.4) performs the actual database round-trip and is

the probe to use when manually confirming that a freshly rolled API container is ready to serve traffic.

Operating-cost analysis. The operating cost decomposes into a fixed infrastructure component and a variable language-model component. The fixed component runs against Amazon Web Services in the **eu-central-1** (Frankfurt) region. At publicly listed June 2026 prices, a single t3.micro Elastic Compute Cloud instance under on-demand Linux billing costs roughly \$0.0114 per hour, or approximately \$8.30 per month at continuous use. A db.t3.micro Relational Database Service instance running PostgreSQL in a single Availability Zone costs roughly \$0.020 per hour or about \$14.60 per month, with an additional \$2.30 per month for the twenty gigabytes of general-purpose SSD storage that the database uses. The Simple Storage Service bucket that holds the CV files draws a per-gigabyte-month charge of about \$0.0245 in this region; for the prototype scale of well under one gigabyte of CV uploads, the monthly bill rounds to a few cents. The front-end bundle is served from a separate bucket through CloudFront, and the egress falls entirely within the always-free tier of one terabyte of data transfer and ten million requests per month. The Systems Manager Parameter Store charges nothing for Standard-tier parameters at the volumes used by Vakansio. The total fixed monthly infrastructure cost is therefore approximately \$25 to \$28 per month once any Free-Tier allowance has been exhausted. While the account remains within the Free-Tier window (the twelve-month allowance for accounts created before the July 2025 Free-Tier restructure, or the equivalent credit allowance afterwards), the realised cost is near zero.

The variable component is the Gemini 2.5 Flash usage, priced at \$0.30 per million input tokens and \$2.50 per million output tokens at the short-context tier. These two rates appear verbatim as the private constants **InputPricePerMillion** and **OutputPricePerMillion** in **CostBreakdown.cs:20-21**, so the application's own cost-estimation arithmetic agrees with the published price list. Per-request cost depends on the pipeline path taken (cold versus warm, judge skip-band hit rate, number of pairs that enter the batched reason generator), and is measured continuously through the **gemini_cost_log** table introduced in Section 3.8 and Section 4.3. Per-stage measured numbers and aggregate per-request statistics are reported in Section 5. Three architectural decisions have the most direct effect on the variable cost. The skip-band in the Composite Judge stage from Section 3.4 avoids approximately thirty to forty per cent of judge invocations. The global-vocabulary path from Section 4.4 replaces per-entity skill expansion with a single batched call. The disabled background workers documented above keep idle Gemini spend at zero.

Operational limits and gaps. Three limits of the current operational posture are worth recording explicitly. First, the deployment uses a single t3.micro host with no horizontal redundancy; the migration path to an Elastic Container Service task definition behind an Application Load Balancer is documented in Section 3.7 and is essentially mechanical from this code base. Second, the manual deployment procedure depends on direct SSH and SCP access to the host; a CI/CD pipeline is not implemented and is recorded as future work in Section 6. Third, no background worker is enabled by default in production; the synchronous fan-out described in Section 4.3 replaces them on the request path, which keeps cost predictable but shifts cold-path latency back onto the user-facing request.

4.7 Selected engineering narratives

This section reports three engineering challenges that were encountered during the implementation of Vakansio and resolved without weakening the layer separation. Each narrative is included because it illustrates a different way the architecture absorbed unexpected change: a low-level Roslyn behaviour that altered how a vocabulary had to be declared in code, a wire-format contract drift between back-end and front-end that surfaced as a silent empty-result bug, and a recurring pattern of graceful degradation that lets the v6 pipeline self-heal across requests.

The Roslyn user-string heap workaround in EvidenceCleaner. The class `Application/Common/Helpers/EvidenceCleaner.cs` owns a small dictionary of “implicit-by-seniority” Product Manager metric tokens — **LTV**, **CAC**, **DAU**, **ROI**, and similar — that are filtered out of the user-visible Missing-skill chips when the vacancy description lists them as separate must-have entries. The natural way to express such a static vocabulary in C# is a collection initializer over a `HashSet<string>`. The Vakansio code base does not use that form. Instead, the vocabulary is stored as a single `const string` with a pipe separator at lines 44–49, and split into the `HashSet` at type-initialisation time. The source comment at lines 40–43 records the reason. On the build host used to ship the deployed image, the conventional collection-initializer form and the corresponding `StringBuilder.AppendLine` literals were observed not to land in the assembly’s `#US` (User String) heap. A single `const string` literal did. The two forms are observationally indistinguishable at the level of behavioural unit tests; the workaround is recorded in code because the constraint that motivated it is not visible at the type level. The lesson is that small low-level compile-time details can change observable artefacts of the assembly, and that capturing the rationale next to the workaround is what allows a future reader to evaluate whether the workaround is still needed when the toolchain changes.

Frontend-back-end contract drift fix series (v6.7.8). The most consequential bug encountered during the second half of the project was a silent empty-result failure on the front-end search page after a routine back-end serialisation change. The change itself was a single line in `API/Program.cs:66`: the addition of `new JsonStringEnumConverter()` to the JSON serialiser’s converter collection, so that the API would emit enum members by name (`"Djinni"`, `"WorkUa"`, `"StrongMatch"`) instead of by their underlying ordinal values (`0`, `1`, `3`). The migration was motivated by Swagger readability and by the front-end’s preference for typed string unions over ordinal magic numbers.

The change broke the front-end in two related but independent places, both fixed under the v6.7.8 label.

The first regression is documented in `frontend/src/pages/JobFeed/JobFeedPage.tsx:153–157`. The job-feed page renders a `<select>` for the source filter whose option values had been numeric strings (`"0"`, `"1"`) under the old contract, and whose filter predicate read `Number(sourceFilter)` to compare against the response payload. After the back-end switch, the response payload carried strings; `Number("Djinni")` returned `NaN` and every comparison of the form `job.source !== sourceFilter` was vacuously truthy because the coerced `NaN` never equals anything. The filter predicate therefore rejected every job, and the user saw an empty list with no error. The fix changed

the option values to mirror the back-end string names directly and removed the **Number()** coercion, so that the comparison is a plain string-against-string equality. The same fix was applied to the legacy **filterRaw** path.

The second regression involved the verdict-label vocabulary. The Domain enum **Verdict** (Section 4.2) defined four members whose CLR names are **StrongMatch**, **PartialMatch**, **WeakMatch**, and **Mismatch**. The string-enum converter therefore emitted these long names on the wire. The front-end **verdictMeta.ts** map, however, keyed verdict-display metadata on the short forms **Strong**, **Partial**, **Weak**, and **Mismatch**. The resolution lives in Domain rather than at the boundary: **VerdictExtensions.ToShortName()** in **Domain/Scoring/Verdict.cs** produces the short form; the DTO uses **ToShortName()** whenever it serialises a verdict; the reasoning prompt continues to use **ToString()** because the prompt vocabulary expects the long names. Two callers, two vocabularies, both routed through a single file. The XML documentation on **Verdict** explicitly records this split.

The composite lesson is that contract-changing edits — anything that alters the wire format — need a cross-stack audit, not a unit test on one side. Single-source-of-truth maps such as **verdictMeta.ts** and **evidenceTier.ts** (Section 3.6) and exhaustive extension methods such as **VerdictExtensions** (Section 4.2) are what made the second fix a one-file edit rather than a scattered touch-up.

Graceful degradation as an architectural pattern. The third narrative is not a single bug but a recurring pattern. The Vakansio v6 pipeline is built on the assumption that any single external call can fail or run out of time, and that the appropriate response is rarely to fail the entire user-facing request. The clearest example is the **SkippedNoAnalysis** field on **GetAggregatedJobsV6Result**. A cold v6 search can return up to roughly one hundred and fifty freshly scraped vacancies, each of which needs a Gemini vacancy-normalisation call before scoring can run. The synchronous fan-out is bounded by **ScoringOptions.SyncNormalizeTimeoutSeconds** (currently three hundred seconds). When the budget blows, the remaining un-normalised vacancies are not scored; they are counted into **SkippedNoAnalysis** and returned alongside the partial result set. On a subsequent request from the same user, the still-unanalysed vacancies in the database are picked up and normalised inside the next budget window. The XML documentation on **ScoringOptions** records this explicitly as a “self-healing” property of the pipeline.

The same pattern reappears in three other places already mentioned in this chapter. The aggregator service (Section 4.4) wraps every individual **IJobSourceService** call in a try-catch so that a single failing source (most often LinkedIn returning a rate-limit response) does not break the entire aggregated search. Every Gemini call carries a short per-call timeout — eight seconds in the inner scoring-loop reason call, fifteen seconds in the Composite Judge, eighteen seconds in the batched Judge, and twenty-five seconds in the batched reason service — and falls back to a deterministic template if the timeout fires or the response fails schema validation, as documented in Section 3.8. The cost-logging path catches every database exception in **CostLogService** (Section 4.4) on the principle that a missing log entry is preferable to a failed user response. The lesson is that the question worth asking at design time for each external call is not whether it will fail but how the system will behave when it does, and that recording the chosen answer in a typed surface such as **SkippedNoAnalysis** makes the degradation observable rather than silent.

5 | Validation

This chapter documents how the Vakansio pipeline was evaluated and what the evaluation found. The chapter has two equally important aims. The first is to describe a defensible evaluation methodology — a decomposition into independent layers, each measured with techniques appropriate to the data type at that layer — that other engineers could rerun and extend. The second is to report the empirical findings honestly, including a primary result on ranking quality that is inconclusive at the conventional significance level on the gold set available within the project’s scope. The decomposition follows the Holistic Evaluation of Language Models framework [6] and adopts large-language-model judging on an independent Anthropic Claude family where reference-free evaluation is required [7].

5.1 Evaluation philosophy and methodology pivot

This section explains how the evaluation strategy reached its current shape. The initial design treated a single end-to-end score-mean-absolute-error metric as the primary evidence for the thesis claim that a Composite Large-Language-Model Judge over the deterministic sub-scores improves match quality relative to a linear weighted sum of the same sub-scores. Two independent audits of that design identified three structural concerns that made the score-error metric unsuitable as the primary endpoint, and the design was revised before the final results were collected. The revised design retains the score-error metric as a calibration diagnostic and adds a top-k ranking-quality metric against independent cross-vendor large-language-model relevance ratings as the primary endpoint.

Why the original score-error metric was demoted. The score-error harness compares the pipeline’s end-to-end output against a deterministic ideal computed by running the same seven sub-score calculators on the gold-curated normalisations of the candidate and the vacancy. The audits identified three structural issues with that comparison. First, the pipeline applies the safety caps and the anti-flag penalty described in Section 3.4, but the ideal score is the uncapped weighted sum; any pipeline that ships caps is therefore mechanically penalised on the metric regardless of whether the cap was appropriate. Second, the gold pair set was stratified by sampling on three primitives — title overlap, seniority match, and skill overlap — that are themselves inputs to the same calculators the Linear pipeline relies on; the test set is therefore engineered to favour the Linear baseline against any pipeline that intentionally departs from the calculator output. Third, the Composite Judge prompt contains rule-precedence clauses that allow the judge to override the documented bounded-adjustment band when an under-qualification, language gap, or family mismatch is detected, and the post-judge clamp does not enforce the bound; this is a real calibration issue, but it is independent of the artefacts above and is more usefully reported as a per-axis diagnostic than as the headline result.

The revised primary endpoint. The revised design adopts top-ten Normalised Discounted Cumulative Gain measured against an independent cross-vendor relevance rating as the primary evidence for the central thesis claim. The configuration uses Anthropic’s

Claude family of models to rate the output of the Google Gemini-backed scoring pipeline; this avoids the within-system circularity of the score-error harness because the rater is from a different vendor family than the model under evaluation. A full multi-vendor judge ensemble — adding judges from a second non-Anthropic family — would strengthen the protocol further and is recorded as future work in Section 5.8. The methodology adopted follows the cross-vendor judging protocol of Zheng et al. [7]: anchor-only ordinal ratings, randomised candidate-presentation order, mandatory per-rating rationale, and stratified batches across verdict buckets. The same protocol underpins the rubric-based evaluation of the bilingual reason text reported in Section 5.5.

What this chapter reports. The remainder of the chapter is organised around the five evaluation layers that emerged from this design. Section 5.2 describes the layered decomposition. Section 5.3 summarises the two structured-output normalisation layers that were frozen before the methodology pivot. Section 5.4 reports the score-error metric as a calibration diagnostic, including the per-axis bias decomposition that surfaces the calibration issue described above. Section 5.5 reports the reason-text-quality layer, including a complete prompt iteration cycle that the eval pipeline guided through four versions. Section 5.6 reports the primary endpoint on top-ten ranking quality, including the per-curriculum-vitae variance behind the headline confidence interval. Section 5.7 traces a single curriculum-vitae and vacancy pair through every layer to show how the metrics compose end-to-end. Section 5.8 names the limitations of the evaluation and the specific extensions that would resolve each of them.

5.2 Layered evaluation architecture

This section presents the layered decomposition of the evaluation. The pipeline produces four distinct artefacts per (curriculum vitae, vacancy) match — a structured curriculum-vitae summary, a structured vacancy analysis, a numeric composite score, and a bilingual reason text — and the central design choice is to evaluate each artefact with the methodology appropriate to its data type rather than to collapse all four into a single end-to-end number. Single-metric evaluation conflates failures across distinct stages and prevents targeted iteration. The decomposition into layers follows the Holistic Evaluation of Language Models framework [6], which establishes multi-axis decomposed evaluation as the appropriate response to the same problem in the broader language-model community. The five-layer table that follows is the road-map for the remainder of the chapter.

Layer	Data type	Gold method- ology	Headline metric	Status
1 — CV normalisation	Structured JSON	Author-curated reference normalisations	Per-field F1 = 0.896 (25 CVs)	Frozen after nine prompt versions
2 — Vacancy normalisation	Structured JSON	Author-curated reference normalisations	Per-field overall = 0.810 (355 of 357 vacancies evaluated)	Frozen after twelve prompt versions
3 — Score	Numeric scalar in $[0, 1]$	Computed ideal from gold inputs	Linear MAE = 0.101, Composite MAE = 0.187 (392 pairs)	Calibration di-agnostic; see Section 5.4
4 — Reason text	Bilingual natural language	Independent-vendor LLM-as-Judge (Claude Opus 4.6)	Overall rubric mean = 8.35 / 10 (391 pairs)	Prompt v6 wins +0.30 over v4 baseline
5 — Ranking	Top-ten ordered list	Independent-vendor LLM-as-Judge (Claude Sonnet 4.6)	$\Delta \text{NDCG@10} = -0.047$, 95% CI $[-0.168, 0.065]$	Inconclusive at $-0.168, 95\% \text{ CI } [-0.168, 0.065]$ curricula vitae

Table 5: The five evaluation layers, the data type each layer measures, the gold-standard methodology used, the headline metric, and the current status.

Layer ownership of failure modes. The five layers partition the pipeline’s failure modes by responsibility. A misclassified vacancy domain is a Layer 2 failure regardless of how the score behaves; a poorly worded reason for an otherwise correctly scored pair is a Layer 4 failure regardless of how the score behaves; a stable score but a degraded top-ten ordering is a Layer 5 failure regardless of how the upstream artefacts behave. Partitioning failure modes by layer is the property that makes targeted iteration possible — when Layer 4 prompt version six (described in Section 5.5) regressed factual accuracy relative to its predecessor without changing the upstream artefacts, the iteration loop could be confined to the Layer 4 prompt and its post-processing.

Layer ownership of methodology choices. Each layer’s gold-standard methodology is the one that the literature treats as standard for its data type. Layers 1 and 2 are structured prediction tasks whose gold-standard methodology is hand-annotated reference outputs and field-level scoring against them. Layer 3 is a numeric regression task whose gold-standard methodology is a computed ideal — the same calculators applied to gold inputs — that requires no subjective interpretation. Layer 4 is a natural-language generation task whose gold-standard methodology, in the absence of human reference texts, is large-language-model judging on an independent Anthropic Claude family on a rubric [7].

Layer 5 is a ranking task whose gold-standard methodology is ordering-quality metrics — Normalised Discounted Cumulative Gain, Spearman’s rank correlation, Kendall’s tau — against per-pair relevance ratings [5] the per-pair ratings themselves come from the same cross-vendor judging protocol.

Layer separation matches the production architecture. The same separation of concerns that Clean Architecture imposes on the production code (Section 3.3) also organises the evaluation. The Domain types that the production scoring service consumes — the gold-curated curriculum-vitae summary, the gold-curated vacancy analysis, the seven sub-score calculators — are exactly the inputs the evaluation tool feeds to each layer. The evaluation tool, packaged as a separate .NET project under **EvalTool/**, runs offline against snapshots of the production gold data; it shares the production code’s interfaces (**IScoringService**, **ISubScoreCalculator**, and others described in Section 4.3) and is therefore guaranteed to test the same code paths that ship to users.

5.3 Layers 1 and 2 — frozen normalisation baselines

This section reports the two structured-output normalisation layers, both of which were closed before the methodology pivot described in Section 5.1 and against whose frozen state the later layers compose. Both layers follow the same shape: a gold set of author-curated reference normalisations, a battery of per-field graders applied to the pipeline output, and an append-only history journal that records every iteration of the prompt or post-processor.

Layer 1 — curriculum-vitae normalisation. The gold set comprises twenty-five curricula vitae, normalised by the author into the same JSON shape that the production pipeline persists as **UserProfile.CvSummary**. Nineteen field-level scored metrics are computed by the graders declared in **EvalTool/Grading/EvaluationEngine.cs**: F1 for skill list comparisons (**technical_skills**, **domain_skills**, **unverified_skills**), Jaccard overlap for target-role lists, exact-match for categorical fields (seniority, work format), and an ordinal-distance grader on the Common European Framework of Reference ladder (A1–C2) for English level. The headline number is the macro-averaged score across these metrics. The current production curriculum-vitae normalisation service reaches an overall score of 0.896 on this gold set after nine prompt versions; the iteration history is preserved in **eval_history/runs.csv**. The layer is considered frozen because subsequent runs change the headline number by less than the per-iteration noise observed across the last three iterations.

Layer 2 — vacancy normalisation. The gold set comprises three hundred and fifty-seven vacancies sampled across the seven source adapters and stratified to cover four heuristic strata (Strong, Partial, Weak, Mismatch) that align with the Composite Judge verdict bucket names. The author normalised each vacancy into the same JSON shape that the production pipeline persists as **JobVacancy.VacancyAnalysisJson**, and seventeen field-level scored metrics are computed by the graders declared in **EvalTool/Grading/VacancyEvaluationEngine.cs**, adapted to the vacancy schema (must-have skill lists, nice-to-have skill lists, domain, seniority, work format, language requirement). The current production vacancy normalisation service reaches an overall field-weighted score of 0.810 across three hundred and fifty-five of the three hundred and fifty-seven gold

vacancies (two were excluded by the loader because of malformed source records) after twelve prompt versions; the iteration history is preserved in `eval_history/vacancy/runs.csv`. The layer is considered frozen on the same noise-threshold criterion as Layer 1.

What the two frozen baselines fix. Because Layers 1 and 2 are frozen, all subsequent layers compose against a known, reproducible upstream. The score and ranking results in Section 5.4 and Section 5.6 are reported against pipeline runs that consume the production normalisation services; the same results computed against the gold normalisations directly (the “ideal” path described in Section 5.4) are reported in parallel as an upper bound on what the pipeline could in principle achieve if its upstream were perfect. The reason-quality results in Section 5.5 are reported against the production normalisation path only, because reason text is generated downstream of scoring and the upper-bound construction does not apply.

What the frozen baselines do not measure. Per-field F1 measures pipeline-output agreement with the author’s gold annotation; it does not measure gold-against-truth correctness. Both gold sets were curated by a single annotator within the project scope, and no inter-annotator coefficient was collected. The fields with lowest F1 — the skill lists, which sit between roughly 0.47 and 0.62 individually depending on the field — are the same fields the downstream scoring formula leans on hardest. The implications of this propagation are revisited in Section 5.8.

5.4 Layer 3 — score as a calibration diagnostic

This section reports the score-error layer and explains why it is retained in the chapter as a calibration diagnostic rather than as the primary evidence for the thesis claim. The layer measures how closely the pipeline’s end-to-end composite score tracks a deterministic ideal computed on gold inputs. The harness was the original primary endpoint of the thesis; the methodology pivot described in Section 5.1 demoted it to a diagnostic role after two independent audits identified three structural issues that make the metric unsuitable for the question the thesis actually asks.

The harness. For every (curriculum vitae, vacancy) pair in a fixed three-hundred-and-ninety-two-pair test set, the harness loads the gold curriculum-vitae summary and the gold vacancy analysis, runs the seven sub-score calculators on those gold inputs, and clamps the weighted sum to $[0, 1]$. The resulting `ideal_score` is the raw weighted sum with no anti-flag penalty, no missing-must-have soft penalty, no language-gap cap, and no family-mismatch cap. The harness then reads the pipeline’s end-to-end output for the same pair and reports the absolute difference. The headline metric, per pair, is `score.mae` the headline metric across pairs is the mean of the per-pair errors. The harness is implemented in `EvalTool/Grading/ScoringEvaluationEngine.cs` and is run through the orchestrator entry point `ScoringEvalOrchestrator.cs`.

The first-look numbers. On the three-hundred-and-ninety-two-pair test set, the Linear pipeline (which sums the seven sub-scores by their configured weights without invoking the Composite Judge) reaches an overall score of 0.899 — mean absolute error of 0.101, verdict-bucket match of sixty-seven point three per cent. The Composite pipeline (which invokes the Composite Judge on every pair and applies the resulting refinement) reaches

an overall score of 0.813 — mean absolute error of 0.187, verdict-bucket match of thirty-one point nine per cent. A naive reading of these numbers is that the Composite Judge is worse than the Linear baseline. The two audits found that reading to be misleading for three independent reasons.

Audit issue one — cap asymmetry. The pipeline applies the safety caps described in Section 3.4 (the floor at 0.20, the ceiling at 0.88, the asymmetric caps on seniority gap, language gap, and family mismatch) and multiplies the result by the anti-flag penalty if applicable. The ideal score applies none of these. A per-pair decomposition of the Linear pipeline’s error reveals that approximately fourteen per cent of pairs carry an active anti-flag and drift by an average of -0.379 against the ideal, while the remaining eighty-six per cent without an anti-flag drift by an average of -0.056 , attributable to the soft missing-must-have penalty and the soft role-family adjustment. Together these account for essentially all of the Linear pipeline’s headline error. The mean absolute error therefore measures the caps, not pipeline quality; any pipeline that ships safety caps is mechanically penalised on this metric.

Audit issue two — selection circularity. The three-hundred-and-ninety-two-pair test set was selected by stratified sampling on three primitives — title overlap, seniority match, and skill overlap — that are also the primary inputs to the Linear pipeline’s **RoleIntentMatchCalculator** and **SkillMatchCalculator**. The stratification weights and verdict-bucket thresholds are recorded in **gold_set_v2/vacancies/selected/selected.json**. The Linear pipeline’s score is essentially a weighted sum of those same primitives, and the ideal against which both pipelines are graded is the same weighted sum. The test set is therefore engineered to make the Linear pipeline hit its own buckets; the Composite Judge, which intentionally departs from the calculator output where the language model disagrees, is judged against a target geometry that is the Linear pipeline’s home turf. The audit estimates that this asymmetry inflates the apparent Linear advantage by roughly 0.02 to 0.04 mean-absolute-error points — between forty and fifty per cent of the visible gap.

Audit issue three — Composite Judge over-penalisation. The Composite Judge prompt states at rule zero that the Judge’s output should remain within ± 0.05 of the input linear score. Rules three, four, and five — the under-qualification cap, the language cap, and the different-family cap — explicitly override rule zero when triggered, and the post-Judge clamp in **GeminiCompositeJudgeService.cs** enforces the $[0, 1]$ range but does not enforce the ± 0.05 band. The five largest Linear-to-Composite divergences on the gold set are downward adjustments of -0.39 , -0.38 , -0.26 , -0.23 , and -0.21 , all on pairs where one of the three override rules fired. This is a real calibration finding, independent of the eval-engine artefacts above. The fix is straightforward and is left for future work: enforce a hard $|s_{\text{judge}} - s_{\text{linear}}| \leq 0.10$ clamp at the boundary of the service.

What remains after the artefacts are subtracted. Of the visible 0.086 gap between the Linear and Composite headlines, the audits attribute roughly 0.04 to the eval-engine artefacts (cap asymmetry plus selection circularity) and roughly 0.05 to the Composite over-penalisation. With both subtracted, the Composite Judge would still underperform the Linear pipeline on the score-tracking metric, but the gap would be roughly half

what the headline suggests, and the remaining gap would be the cost of the deliberate Composite departure from the calculator output.

Empirical confirmation of the calibration attribution. The audit’s attribution of approximately 0.05 of the score-tracking gap to the rule-precedence override is confirmed empirically in Section 5.6. Running the persisted Composite scores through a post-hoc ± 0.10 clamp against the Linear score collapses the largest negative outlier from a per-curriculum-vitae delta of approximately -0.54 to essentially zero. The mechanism predicted by the audit and the mechanism observed in the ranking-quality experiment are the same.

Why this metric answers the wrong question. The deeper point of the audits is that score-tracking is the wrong primary endpoint for the thesis claim. The thesis claim is about ranking quality — whether the top-ten candidate list produced by the Composite pipeline is better-ordered against human-judged relevance than the top-ten list produced by the Linear pipeline. A pipeline can have a higher mean absolute error from the linear-ideal while producing a top-ten ordering that aligns better with downstream judgement of fit. The primary endpoint was therefore replaced; the result on the new endpoint is reported in Section 5.6. The score-tracking harness is retained in the evaluation tool because the per-sub-score bias decomposition it produces — for example, a per-axis bias of $+0.028$ on **DomainAlignment** and $+0.045$ on **RoleIntentMatch** — is a useful diagnostic when a sub-score calculator is rebalanced.

5.5 Layer 4 — reason-text quality and the prompt iteration cycle

This section reports the reason-text-quality layer, which evaluates the bilingual English and Ukrainian explanations that the pipeline generates for the top-thirty candidate vacancies in every result. The data type here is short-form natural-language text, for which the field-grader methodology used in Layers 1 and 2 does not apply. The methodology adopted is large-language-model judging on an independent Anthropic Claude family on a structured rubric, following the protocol established by Zheng et al. for evaluating language-model output without human reference texts [7] and the domain-specific rubric design proposed by Hu et al. for evaluating hiring explanations specifically [8].

The judge. The judge model is Claude Opus 4.6 from Anthropic. The choice of an Anthropic judge avoids the within-system bias that a same-family judge would carry per Zheng et al.: the model whose outputs are being rated is Google’s Gemini 2.5 Flash, so the judge and the system under test come from different vendor families. The judge is the strongest model in the Anthropic family at the time of the evaluation; the choice of the strongest tier on the judge side is recommended by Zheng et al. for the rating side of an asymmetric configuration. The single-judge-family configuration is a deliberate scope choice rather than a methodological end-state: a full multi-vendor ensemble that adds judges from a second non-Anthropic family is recorded as the principal limitation in Section 5.8 and as future work in Section 6.

The rubric. Each (candidate, vacancy) reason text is rated on six dimensions. Four are ordinal on the six-point anchor set $\{0, 2, 4, 6, 8, 10\}$: **factual_accuracy** (do the strengths, gaps, and context claims correspond to the actual curriculum-vitae and vacancy

content?), **completeness** (are the top factual signals — both positive and negative — surfaced?), **specificity** (is the phrasing pair-unique rather than generic boilerplate?), and **relevance_for_user** (is the explanation actionable for a candidate’s apply-or-skip decision?). Two are binary: **verdict_match** (does the verdict word agree with the score bucket?) and **uk_term_preservation** (are technical brand-and-tool tokens preserved in Latin script in the Ukrainian half?). The rubric, the anchor descriptions for each ordinal value, and worked examples are recorded in `docs/research/reason_eval_rubric.md`.

Bias mitigation in the judging protocol. Five mitigations are applied per Zheng et al. The candidate identifiers presented to the judge are anonymised at random (candidate A versus candidate B with a random mapping back to the actual prompt version under evaluation). The judge is instructed not to compare candidates against one another within a pair, only against the rubric. The ordinal dimensions use anchor-only scoring with no intermediate values, which forces the judge to commit to a clear bucket. Pairs are organised into batches of twenty-eight stratified proportionally across the four score-derived verdict buckets, so that no batch consists exclusively of one bucket. The judge is required to emit a per-rating rationale alongside each numeric score, both for human inspection and for downstream auditability.

Judge self-consistency check on the pilot. Before the main rating runs were collected, two independent calls to the Claude Opus 4.6 judge — with randomised pair-presentation order and non-zero sampling temperature — rated the same ten pilot pairs, and Krippendorff’s α was computed across the ordinal and binary dimensions of the rubric. The resulting α measures the judge’s stochastic self-consistency at the chosen sampling configuration; it is not an inter-vendor inter-rater coefficient. The pilot-gate threshold adopted for this self-consistency check is $\alpha \geq 0.6$ for the ordinal dimensions and $\alpha \geq 0.7$ for the binary dimensions, below the conventional Krippendorff (2004) tentative-conclusion threshold of 0.667; the lower threshold is justified because the check is intended to detect catastrophic instability of the judge rather than to validate inter-rater reliability in the literature sense. Mean ordinal α on the pilot was 0.695 with the binary raw agreement between ninety-three and ninety-seven per cent. The lowest per-dimension ordinal α — **factual_accuracy** at $\alpha = 0.603$ — was within 0.003 of the adopted threshold and is reported here for transparency. An inter-vendor inter-rater coefficient computed across judges from different model families is recorded as the principal limitation in Section 5.8.

Author spot-check of the gold ratings. The judge ratings are not treated as a black box. The author drew a stratified sample of pairs across the four verdict buckets and read each pair end-to-end: the curriculum vitae, the vacancy, the judge’s numeric score on every dimension, and the per-rating rationale. The purpose of the spot-check is a plausibility audit, not a primary annotation pass — the judge’s ratings remain the gold values used in the statistical inference. On the spot-checked pairs, no rater-flagged bias pattern was recorded; this is an absence-of-evidence claim made under a non-blind, single-rater protocol and is not advanced as evidence of judge correctness. Pairs where the spot-check disagreed with the judge were noted for future iteration but were not retroactively adjusted, because adjustment by an interested author would itself be a source of bias. The author is not a disinterested rater — the project under evaluation is the author’s own —

and the spot-check is reported here for transparency rather than as an independent gold standard.

Statistical inference protocol. The primary endpoint for each prompt-version comparison is the paired difference of the per-pair overall mean across all six dimensions. Confidence intervals are produced by paired bootstrap with ten thousand resamples. Per-dimension deltas are reported as exploratory.

The prompt iteration cycle. The thesis demonstrates a complete detect-fix-validate-iterate loop through four prompt versions, each developed in response to a regression that the previous iteration surfaced. Table 6 summarises the comparison endpoints; the per-version narrative is given in the paragraphs that follow.

Comparison	Δ overall	95% CI	Significant $\alpha = 0.05$	at	Headline signal
v4 vs v2	+0.55	[+0.42, +0.68]	Yes		Context enrichment works, factual regression detected
v5 vs v4	-0.03	[-0.14, +0.08]	No		Targeted fix introduces a recall trade-off
v6 vs v4	+0.30	[+0.19, +0.40]	Yes		Templates resolve trade-off across all four ordinal dimensions

Table 6: Paired-bootstrap overall reason-quality deltas across the three head-to-head studies. Each study uses approximately 391 pairs, anonymised, with the protocol described in this section.

v4 versus v2 — context enrichment works, but factual accuracy regresses. The v4 prompt added context-lead phrasing for the salient cases of overqualification, cross-domain candidacy, and role-family mismatch, together with a computed **ReasonContext** record produced by **ScoringServiceV2.BuildReasonContext()** and a post-generation **ReasonValidator** safety net. The overall improvement over the v2 baseline is +0.55 on the zero-to-ten scale (95 per cent paired-bootstrap CI strictly excluding zero). The eval pipeline simultaneously surfaced a regression in **factual_accuracy** of -0.43. Investigation of the regressed pairs traced the regression to the language model inferring seniority and domain labels from its own judgement rather than from the computed **ReasonContext** fields.

v5 versus v4 — the targeted fact-fix introduces a precision-recall trade-off. The v5 prompt rewrote the context-lead rule to forbid inference and to require strict referencing of the **ReasonContext** field names, together with explicit anti-examples of the observed fabrication patterns. The targeted dimension responded — **factual_accuracy** improved by +0.71 — but at the cost of significant regressions in **completeness** (-0.62) and **relevance_for_user** (-0.37). The overall delta is statistically indistinguishable from

zero. The regression mechanism, on inspection of the rationale text, is the prompt becoming too cautious: when the language model was uncertain whether a context flag applied, it omitted the lead entirely, dropping signals that the user would have found useful.

v6 versus v4 — the template-based resolution. The v6 prompt replaced the free-form context generation with four constrained templates — Role-Family Mismatch, Cross-Domain, Overqualification, Underqualification — that interpolate the literal **ReasonContext** field values into a fixed sentence shape, ordered by user-actionability priority, and bounded by a word-budget rule. The intuition is that constrained templates both prevent fabrication (because the language model interpolates given field values rather than generating new claims) and ensure inclusion (because the template fires when the flag is set rather than relying on the model to decide). The result is significant improvement on all four ordinal dimensions: **factual_accuracy** +0.50, **completeness** +0.23, **specificity** +0.77, **relevance_for_user** +0.31, and an overall delta of +0.30 with a 95 per cent confidence interval strictly excluding zero. The v6 prompt is the production version at the time of writing.

What the iteration cycle demonstrates beyond the headline. The three head-to-head studies show the evaluation pipeline detecting a regression (v4’s factual-accuracy drop), validating a targeted fix (v5’s factual-accuracy recovery), detecting the resulting recall trade-off (v5’s completeness and relevance regressions), and finally validating a refactor that resolves the trade-off (v6’s across-the-board improvement). Each detection step was a confidence-interval calculation, not an eyeball. Each fix step was a single component change (prompt rule, then templates) with the rest of the pipeline held fixed. The iteration cycle is itself a contribution of the thesis to the methodology of prompt-driven language-model-based system engineering.

5.6 Layer 5 — top-ten ranking quality (the primary endpoint)

This section reports the primary endpoint of the thesis: top-ten Normalised Discounted Cumulative Gain (NDCG@10) [5] of the Composite pipeline against the Linear baseline, measured on a per-(curriculum vitae, candidate-set) basis with paired-bootstrap confidence intervals. The endpoint is the one the original thesis claim is actually about — whether the Composite pipeline produces a better-ordered top-ten candidate list than the Linear pipeline — and was adopted after the methodology pivot described in Section 5.1.

The judge. For this layer the judge model is Claude Sonnet 4.6 from Anthropic, distinct from the Claude Opus 4.6 judge used in Section 5.5. The choice of the more cost-effective tier reflects the task: each per-pair rating is a single ordinal value on the anchor set {0, 2, 4, 6, 8, 10} rather than the six-dimension rubric that Layer 4 requires, and the ratings are needed at scale (three hundred and ninety-one pairs across fourteen curricula vitae). The cross-vendor configuration is preserved — an Anthropic judge rates the output of the Gemini-backed Vakansio pipeline — and the same anchor-only ordinal protocol with mandatory per-rating rationale and randomised candidate order is applied. The single-vendor judge limitation is named explicitly in Section 5.8.

The rated quantity. The judge rates each (curriculum vitae, vacancy) pair on a single dimension named `match_quality`. The anchor descriptions span the range from “no signal of fit between the candidate and the vacancy” at zero to “the candidate is an exceptional match for the role as described” at ten. Pairs are presented to the judge with the full curriculum-vitae summary and vacancy analysis but without either pipeline’s score; the rating is therefore a relevance signal independent of either system under test.

Author spot-check of the match-quality ratings. The match-quality gold set was subjected to the same plausibility audit described in Section 5.5: the author drew a stratified sample of pairs across the four verdict buckets and read each pair end-to-end against the supporting evidence. The spot-check found no rater-flagged bias pattern — an absence-of-evidence claim under the same non-blind single-rater limitation acknowledged in Section 5.5. The audit is reported for transparency rather than as an independent gold standard; the author’s interest in the outcome is acknowledged in Section 5.8.

The endpoint and the test. For each of the fourteen curricula vitae, the candidate vacancy set (approximately twenty-eight vacancies per curriculum vitae, stratified across verdict buckets) is independently ranked by the Composite pipeline and by the Linear pipeline, and the top ten of each ranking is scored against the judge’s match-quality ratings using the standard NDCG@10 definition with $\log_2$ discounting. The per-curriculum-vitae primary statistic is the difference between the Composite NDCG@10 and the Linear NDCG@10. The aggregate statistic is the mean of the per-curriculum-vitae differences, with a 95 per cent paired-bootstrap confidence interval over the fourteen curricula vitae (ten thousand resamples).

The headline result. The mean of the per-curriculum-vitae differences is $\Delta \text{NDCG@10} = -0.047$, with a 95 per cent paired-bootstrap confidence interval of $[-0.168, +0.065]$. The interval includes zero. Cohen’s d on the per-curriculum-vitae differences is -0.199 , a small directional effect favouring the Linear pipeline. The per-curriculum-vitae outcome is split as Composite winning on five curricula vitae, Linear winning on nine, and no ties. The honest reading of these numbers is that the central thesis claim that the Composite Judge improves ranking quality is neither supported nor refuted on this gold set at the conventional significance level; the data warrant the description “directionally non-significant” rather than either “wins” or “loses”.

Both pipelines achieve strong ranking quality on the full ordering. The complementary full-ordering correlation against the judge’s match-quality ratings is strong for both pipelines. Per-curriculum-vitae mean Spearman’s rank correlation is $\rho = +0.690$ for the Composite pipeline and $\rho = +0.691$ for the Linear pipeline; Kendall’s τ is $+0.481$ and $+0.503$ respectively. Both numbers are an order of magnitude above the level a random ranker would produce and confirm that the underlying scoring methodology — deterministic sub-scores composed with weights informed by the requirements in Section 2.7 — produces useful orderings regardless of the Composite Judge stage. The Composite and Linear pipelines are statistically indistinguishable on the full-ordering correlation as well.

Per-curriculum-vitae variance is high. The aggregate NDCG@10 delta hides substantial per-curriculum-vitae variance. The Composite pipeline wins on cases where qualitative judgement materially changes the ordering — a senior quality-assurance automation

curriculum vitae against a hand-crafted set of borderline candidates ($\Delta = +0.331$), a frontend-engineer curriculum vitae against React-versus-Angular-mixed candidates ($\Delta = +0.158$), and the author’s own career-switching product-manager curriculum vitae against a heterogeneous candidate set ($\Delta = +0.199$). The Linear pipeline wins on cases where the deterministic skill-overlap signal is decisive and a Composite adjustment introduces noise — a broad full-stack engineer curriculum vitae against generic full-stack roles where the Composite Judge over-penalises depth specialisation ($\Delta = -0.643$), an engineering-manager curriculum vitae ($\Delta = -0.263$), and a senior product-manager curriculum vitae ($\Delta = -0.172$). The single broad-full-stack pair (-0.643) dominates the per-curriculum-vitae arithmetic; with that outlier removed the mean delta drops to approximately -0.001 , statistically indistinguishable from a tied result. The outlier is not removed from the headline because removing inconvenient data is itself a methodological hazard, but it is reported transparently here so that the reader can judge.

What the calibration defect from Section 5.4 contributes. The Composite Judge over-penalisation on the under-qualification, language-gap, and family-mismatch rules described in Section 5.4 is one identifiable mechanism behind the observed negative cases. The full-stack curriculum vitae that drives the headline delta downward triggers the family-mismatch rule on several of the narrower full-stack candidates in its set, and the resulting downward adjustment of the Composite score moves those candidates out of the top-ten window. The empirical contribution of the defect is quantified in the next paragraph.

Post-hoc sensitivity check under the proposed calibration clamp. The fix proposed in Section 5.4 is a hard ± 0.10 clamp at the boundary of `GeminiCompositeJudgeService`. The clamp is a pure post-processing filter on the Judge’s output. The Composite score and the Linear score are both already persisted per pair in the evaluation results. The same arithmetic the proposed code patch would apply at production time can therefore be applied analytically to those persisted scores, and re-running the Gemini Judge calls is not required because the clamp does not change the model’s internal decision but only the value the boundary returns. The post-hoc clamp is applied per pair as $s'_{\text{composite}} = \max(s_{\text{linear}} - 0.10, \min(s_{\text{linear}} + 0.10, s_{\text{composite}}))$, and NDCG@10 is recomputed against the unchanged Sonnet match-quality ratings. The sensitivity analysis reproduces the published headline (mean $\Delta = -0.047$) within rounding as $\Delta = -0.040$. Under the clamp the mean per-curriculum-vitae delta moves to $\Delta = +0.018$, with a 95 per cent paired-bootstrap confidence interval of $[-0.054, +0.091]$ that still crosses zero. The directional sign of the headline flips from negative to positive, but neither the original nor the clamped interval excludes zero, so the verdict of “directionally non-significant” is robust to the defect’s removal. The largest negative outlier, the broad-full-stack curriculum vitae `17_realformat_swe_fullstack`, drops from a per-curriculum-vitae delta of approximately -0.54 in the original analysis to $\Delta \approx 0$ under the clamp, empirically confirming the audit’s attribution of the outlier to the rule-precedence over-penalisation. Per-curriculum-vitae outcome shifts from a split of five Composite wins versus nine Linear wins (original) to a split of seven each (clamped). The fix itself is recorded as future work in Section 6 because deploying it requires re-running the production pipeline; the

result reported here is that the methodology’s verdict is independent of whether the fix is deployed.

What this layer does and does not claim. The layer does not claim that the Composite Judge improves ranking quality over the Linear pipeline. The layer does not claim the opposite either. The interval crosses zero. What the layer does claim is that the methodology — cross-vendor language-model judging on an anchor-only relevance scale, NDCG@10 with paired-bootstrap inference, per-curriculum-vitae variance reporting, and explicit attribution of the largest observed negative case to a fixable calibration defect — is a defensible answer to the question the thesis set out to ask. The current answer is honestly inconclusive on this gold set; the conditions under which the answer would change are stated in Section 5.8.

5.7 End-to-end worked example

This section traces a single (curriculum vitae, vacancy) pair through all five evaluation layers so that the reader can see how the metrics compose end-to-end. The pair was selected because it is one of the cases on which the Composite pipeline wins materially over the Linear pipeline — illustrating the qualitative judgement the Composite Judge is meant to provide — even though the aggregate result reported in Section 5.6 is inconclusive. Reporting an aggregate-negative result alongside a per-curriculum-vitae positive case is itself the kind of disaggregation the layered methodology is meant to enable.

The chosen pair. The curriculum vitae is **15_qa_senior_automation** — a senior quality-assurance automation engineer with approximately eight and a half years of experience, an English level of B2, and a technical stack centred on Python, Selenium, Playwright, Cypress, and JMeter. The vacancy is **e66e80c0**, a senior application-programming-interface and back-end performance and load testing role requiring exactly that Python and JMeter stack. The author’s manual reading of the pair places it at the top of the candidate set; the judge agrees, rating it at the maximum value of ten on the **match_quality** scale used in Layer 5.

Layer 1 — the curriculum-vitae normalisation. The production pipeline emits the curriculum-vitae summary as a structured JSON shape that captures multiple fields — among them **seniority**, **english_level**, **experience**, **technical_skills**, **domain_skills**, and **target_roles**. For this curriculum vitae, the gold normalisation records **seniority = senior**, **english_level = B2**, an experience history whose durations sum to approximately one hundred and seven months (eight years and eleven months), and skill lists including **Python**, **Selenium**, **Playwright**, **Cypress**, **pytest**, and **JMeter**. The production pipeline reproduces these fields at or near the layer-level mean reported in Section 5.3.

Layer 2 — the vacancy normalisation. The vacancy gold normalisation records **seniority_required = senior**, an English requirement of B2, and a must-have skill list including **Python**, **API testing**, **JMeter**, and **Gatling**. The production vacancy normalisation reproduces this shape at or near the layer-level mean reported in Section 5.3. The two normalised artefacts (curriculum vitae and vacancy) are the inputs to the scoring pipeline.

Layer 3 — the score. The Linear pipeline computes a weighted sum of the seven sub-scores on this pair and returns 0.642. The Composite pipeline runs the same computation, invokes the Composite Judge on the result, and returns 0.680 after the Judge applies a small upward refinement reflecting the strong qualitative match. The deterministic ideal computed from the gold inputs (the metric described in Section 5.4) is in the same neighbourhood. As discussed in Section 5.4, this number is reported here as a calibration diagnostic rather than as the headline result; the difference between Linear and Composite on this pair is the kind of refinement the Composite Judge is intended to provide.

Layer 4 — the reason text. The v6 prompt produces a bilingual reason text for this pair that begins with a context-lead phrase recognising the strong fit (“Strong match: senior automation experience with the requested stack”), followed by a **Strengths** section that lists the matched skills (Python, JMeter, API testing, Selenium for cross-context familiarity), a **Gaps** section that surfaces the minor missing items (Gatling specifically, though closely adjacent to JMeter), and a **Recommendation** section that names the apply-or-skip signal for the candidate. The Ukrainian half preserves the technical brand-and-tool tokens (Python, JMeter, Gatling, Selenium) in Latin script per the **uk_term_preservation** dimension of the Layer 4 rubric. On the Layer 4 rubric, this pair scores at or near the maximum on every ordinal dimension because the strengths claim is verifiable against the curriculum vitae and the missing-must-have claim is verifiable against the vacancy.

Layer 5 — the ranking position. The worked example uses two complementary NDCG@10 measurements on the same curriculum vitae for pedagogical clarity. The first is a like-for-like paired-subset comparison restricted to the nineteen candidate pairs on which both pipelines emit a non-corrupted score (the Composite-pipeline pass had eleven scoring failures on the original thirty-pair set). On that paired subset the Composite pipeline orders the chosen vacancy at position five and the Linear pipeline orders it at position eight, with the Composite NDCG@10 reaching 0.564 against the Linear pipeline’s 0.442 — a per-curriculum-vitae Δ of +0.122. The second is the headline NDCG@10 on the full thirty-candidate set, which uses each pipeline’s full candidate coverage; on that set the Linear pipeline ranks the gold-ten vacancy out of the top-ten window altogether (at position eleven), the Linear NDCG@10 falls to 0.230, the Composite NDCG@10 remains at 0.561, and the per-curriculum-vitae Δ widens to +0.331. The two numbers measure different quantities — a like-for-like comparison and a coverage-sensitive comparison respectively — and the $\Delta = +0.331$ figure is the one that contributes to the aggregate reported in Section 5.6.

Composite rank	Composite score	Gold match_quality	Note
1	0.808	4	Over-ranked by Composite
2	0.780	8	
3	0.730	8	
4	0.730	8	
5	0.680	10	The pair traced in this section
6	0.660	4	
7	0.630	4	
8	0.600	4	
9	0.588	4	
10	0.588	2	

Table 7: Top-ten Composite ranking for the curriculum vitae **15_qa_senior_automation** on the paired-subset analysis described in the surrounding paragraphs. The pair traced through this section is at rank five; the Linear pipeline places the same pair at rank eight on the same paired subset.

What the worked example shows. The example demonstrates the property the layered methodology is designed to deliver: a pair on which the Composite pipeline does what the Composite Judge is designed to do — promote a strongly relevant candidate that the deterministic calculator alone would underweight — surfaces as a positive Δ at Layer 5 even though the aggregate Δ across all fourteen curricula vitae is inconclusive at the same significance level. Aggregate non-significance does not imply per-curriculum-vitae uniformity; the layered methodology makes this visible rather than averaging it away.

5.8 Threats to validity

This section names the limitations of the evaluation and the specific extensions that would resolve each of them. The order is from the most consequential to the least.

Curriculum-vitae sample size. The primary endpoint on Layer 5 is computed over fourteen curricula vitae. The 95 per cent paired-bootstrap confidence interval reported in Section 5.6 — $[-0.168, +0.065]$ — is wide enough that the chapter’s headline of “directionally non-significant” is the strongest description the data support. A larger gold set would either tighten the interval to exclude zero in either direction or confirm the per-curriculum-vitae variance pattern documented in Section 5.6. Per-family claims at the resolution of a single role family (Engineering, Data, Product, Design) are descriptive in this thesis rather than inferential, because no family has the per-family $n \geq 30$ that an inferential paired test would require.

Outlier influence on the headline delta. The single broad-full-stack curriculum vitae (**17_realformat_swe_fullstack**) contributes a per-curriculum-vitae delta of -0.643 , which dominates the aggregate arithmetic. With that single curriculum vitae removed,

the mean delta drops to approximately -0.001 — statistically indistinguishable from a tied result. The outlier is retained in the headline because removing inconvenient data is itself a methodological hazard, but the reader is invited to weigh the headline accordingly.

Composite Judge calibration defect. The over-penalisation pattern documented in Section 5.4 is the largest identifiable mechanism behind the observed negative cases on Layer 5. A hard $|s_{\text{judge}} - s_{\text{linear}}| \leq 0.10$ clamp at the boundary of **GeminiCompositeJudgeService** would isolate the Composite Judge’s qualitative contribution from this artefact. The fix has not been applied in production in time for the current evaluation, but a post-hoc sensitivity check is reported in Section 5.6. Applying the clamp arithmetically to the persisted Composite scores moves the mean delta from $\Delta = -0.040$ (the sensitivity analysis’s reproduction of the published -0.047 headline within rounding) to $\Delta = +0.018$, and collapses the largest negative outlier from approximately -0.54 to essentially zero. Both the pre- and post-clamp confidence intervals still cross zero, so the methodology’s verdict of inconclusive at $\alpha = 0.05$ is robust to whether the calibration fix is deployed; the production deployment of the fix is recorded as future work in Section 6.

Single-vendor judge family. All judge ratings on Layer 4 and Layer 5 originate within the Anthropic Claude family — Claude Opus 4.6 on Layer 4 and Claude Sonnet 4.6 on Layer 5. The intra-vendor configuration was a deliberate choice: it allowed a single rating pipeline and prompt format to be reused across the two layers, and the two model tiers were selected to match each layer’s task (the stronger Opus for the multi-dimensional rubric, the more cost-efficient Sonnet for the single-dimension anchor-only scale at the scale of 391 pairs). The inter-vendor extension — adding judges from at least one other vendor family, for example a model from OpenAI’s GPT family and a model from a third vendor such as Mistral’s Large tier — would strengthen the external validity of the cross-vendor protocol of Zheng et al. [7] and would allow inter-judge agreement to be measured directly. A Gemini Pro judge is not a suitable addition for this extension because Gemini Pro and the production Gemini 2.5 Flash share the same vendor family, which is the configuration the cross-vendor protocol is designed to avoid. The extension is recorded as future work in Section 6 and was not applied in time for the current evaluation because of access constraints at the time of submission.

Single-annotator gold sets in Layers 1, 2, and the author spot-checks. The curriculum-vitae and vacancy gold normalisations were curated by a single annotator (the author). The author spot-checks of the Layer 4 and Layer 5 ratings are plausibility audits under a non-blind protocol rather than independent annotations, and the author is not a disinterested rater on either path. Inter-annotator agreement coefficients on a subset of the gold normalisations and on a subset of the match-quality ratings — by an annotator with no interest in the outcome — would be the appropriate extension; both are recorded as future work in Section 6.

Judge-in-the-loop prompt iteration. The Layer 4 prompt iteration cycle reported in Section 5.5 used the Claude Opus 4.6 judge both during prompt development (as the signal the author iterated against) and during the final v6-versus-v4 comparison. This is a construct-validity threat per Messick: the v6 prompt may be better at producing reasons that satisfy the specific stylistic priors of one Anthropic checkpoint rather than reasons

that humans find more useful. The chapter’s defence — that the v6 templates impose hard structural constraints on the output (literal field interpolation, fixed sentence shapes, priority ordering) rather than optimising free-form judge preferences — is supported by the per-dimension improvements being mechanistic (factual accuracy from forbidden inference, specificity from **vacancy_role_short** interpolation) rather than stylistic. An out-of-loop human comparison of the v6-versus-v4 deltas, or a comparison judged by an independent vendor family, is the appropriate confirmatory study and is recorded as future work.

Naive sub-score calculators. The deterministic sub-score calculators described in Section 3.4 use canonical-alias intersection and Jaccard overlap rather than embedding-based semantic similarity. A vacancy that requires Kubernetes and a curriculum vitae that lists Docker, Helm, and OpenShift score lower than they should on the skill axis, regardless of the obvious semantic equivalence. The Composite Judge is the design response to this limitation — the language model can recognise the equivalence — but the residual on the Linear baseline is bounded by the calculator’s surface-form fidelity.

Production deployment validation. The v6 prompt was deployed to production at the time of submission; an A/B comparison against the v4 prompt under real-user click-through rates has not yet been collected. The reason-quality eval is a strong predictor of user satisfaction in the cross-vendor judging literature, but it is not user satisfaction directly.

What would change the answer. Three changes, in combination, would be sufficient to resolve the central thesis claim definitively. First, an extension of the curriculum-vitae gold set to at least thirty per role family would tighten the Layer 5 confidence interval enough to support per-family inference. Second, the Composite Judge calibration fix would remove the largest identifiable confound in the headline. Third, a multi-vendor judge ensemble (Anthropic Claude, an OpenAI model at its strongest tier, and a Google Gemini Pro judge) would establish that the result is not an artefact of the single judge family chosen for the current evaluation. None of the three is conceptually difficult; all three are recorded as future work in Section 6.

6 Conclusion

6.1 Project summary

This thesis presented Vakansio, a deployed personalised job-matching service for the Ukrainian information-technology market, together with the evaluation methodology and empirical results that accompany it. The system aggregates vacancies from seven external sources, normalises them and the candidate’s curriculum vitae into structured representations, scores every candidate–vacancy pair through a deterministic seven-axis sub-score calculator refined by a Composite Large-Language-Model Judge, and presents the result with bilingual explanations. The deployed instance runs on a single Amazon Elastic Compute Cloud t3.micro host behind Caddy, with managed PostgreSQL, object storage, and content-delivery components. It is reachable at dsus1dizgh006.cloudfront.net. The implementation follows the Clean Architecture decomposition [2] with the SOLID principles mapped to concrete code artefacts.

The evaluation decomposes the pipeline into five layers — curriculum-vitae normalisation, vacancy normalisation, score, reason text, and ranking — and applies methodology appropriate to each layer’s data type. The decomposition follows the multi-axis principle of the Holistic Evaluation of Language Models framework [6]. The two normalisation layers are frozen at per-field scores of 0.896 and 0.810 respectively. The score layer is reported as a calibration diagnostic following two audits that identified structural artefacts in its harness. The reason-text layer reports a significant improvement on all four ordinal rubric dimensions for the v6 prompt over the v4 baseline (overall $\Delta = +0.30$, 95 per cent paired-bootstrap CI $[+0.19, +0.40]$, $N \approx 391$ pairs), judged by Claude Opus 4.6 on the protocol of Zheng et al. [7]. The ranking layer is the primary endpoint of the central thesis claim. It is reported as directionally non-significant on the available gold set of fourteen curricula vitae (Δ NDCG@10 = -0.047 , 95 per cent paired-bootstrap CI $[-0.168, +0.065]$). A post-hoc sensitivity check on the proposed calibration clamp shifts the headline to $\Delta = +0.018$ but does not change the inconclusive verdict at the conventional significance level.

6.2 Comparison with the initial objectives

The four explicit objectives stated in Section 1.3 are revisited below.

Objective one — a working public service. The system runs on the open internet, supports user registration and login, accepts a curriculum-vitae upload as a PDF, fetches vacancies from seven external sources, and returns a personalised list with verdicts and bilingual explanations. The deployment topology and operational posture are documented in Section 3.7 and Section 4.6. The objective is met.

Objective two — a disciplined software architecture. The code base is organised as four .NET projects (Domain, Application, Infrastructure, API) whose project-reference graph enforces the Clean Architecture dependency rule at build time. The Domain project declares zero package references in `Domain/Domain.csproj` the Application project references Domain only; the Infrastructure project references Application (and Domain

transitively); the API project references Application and Infrastructure. The SOLID principles are mapped onto specific code artefacts in Section 3.3 and elaborated through Section 4.2, Section 4.3, and Section 4.4. The objective is met.

Objective three — measurable match quality. The five-layer evaluation methodology described in Section 5.2 is the architectural answer to this objective. The empirical answer is mixed and honestly so. On the reason-text layer the v6 prompt achieves a statistically significant improvement over the v4 baseline across all four ordinal rubric dimensions. On the ranking layer the central thesis claim that the Composite Judge improves ranking quality over the Linear baseline is neither supported nor refuted at the conventional significance level on a gold set of fourteen curricula vitae; both pipelines achieve strong full-ordering correlation against the cross-vendor relevance ratings (Spearman $\rho \approx 0.69$ for each). The conditions under which the ranking result would be resolved definitively are stated in Section 5.8 and revisited in Section 6.4. The methodological component of the objective is fully met; the empirical component is partially met and openly characterised.

Objective four — transparency of every large-language-model call. Every Gemini call writes a row to the `gemini_cost_log` table through the per-stage cost accumulator described in Section 3.8 and implemented at `Application/Common/Diagnostics/CostBreakdown.cs`. The accumulator records the stage, the call count, the input and output token counts, the wall-clock duration, and the dollar cost computed from the current Gemini 2.5 Flash short-context tier prices. The administrative endpoints described in Section 3.8 expose the data for offline querying. The objective is met.

6.3 Encountered difficulties

This section records five difficulties whose resolution shaped the chapters above. Each is documented in detail in the chapter cited.

Mid-thesis evaluation-methodology pivot. The originally planned primary endpoint — a single end-to-end score-mean-absolute-error metric — was identified by two audits as structurally biased against any pipeline that deliberately departs from the deterministic calculator output, for the reasons documented in Section 5.4. The pivot to top-ten Normalised Discounted Cumulative Gain against independent-vendor relevance ratings, with the original metric retained as a calibration diagnostic, was the engineering response. The pivot itself is treated in the chapter as a methodological contribution rather than as a hidden failure.

Roslyn user-string heap workaround. A small Application-layer vocabulary in `EvidenceCleaner.cs` had to be declared as a single `const string` with a pipe separator and split at type initialisation, rather than as a collection initialiser. The reasons are documented in Section 4.7. The workaround is recorded in the source code so that a future toolchain change can revisit its necessity.

Front-end and back-end contract drift. A back-end serialisation change in version 6.7.8 (the addition of `JsonStringEnumConverter` to the JSON converter list) broke the front-end source filter and the verdict-label vocabulary at the same time. The mechanism of the silent empty-result failure and the two-file fix are documented in Section 4.7. The

episode is the strongest argument for the single-source-of-truth maps used on the front-end (Section 3.6).

Composite Judge calibration defect identified but not deployed. The rule-precedence override pattern that allows the Composite Judge to depart from its documented bounded-adjustment band is documented in Section 5.4, and the proposed fix is a single-file change at the boundary of `GeminiCompositeJudgeService`. The fix was not deployed in time for the submitted evaluation. The post-hoc sensitivity check reported in Section 5.6 quantifies the defect’s contribution and confirms that the methodology’s verdict on the ranking layer is robust to whether the fix is deployed.

Single-vendor judge constraint at submission time. The full cross-vendor judge ensemble — a Claude judge plus an OpenAI judge plus a third-vendor judge — was constrained at submission time by access to the second-vendor application-programming-interface. The intra-Anthropic ensemble adopted (Claude Opus 4.6 for the reason-text layer, Claude Sonnet 4.6 for the ranking layer) is reported as a deliberate scope choice; the inter-vendor extension is recorded in Section 5.8 and revisited below.

6.4 Future perspectives

This section names the extensions that would most directly tighten the empirical findings reported above. Each is a discrete piece of work with a clearly defined cost and a known location in the code base or the gold set.

Production deployment of the calibration clamp. The hard $|s_{\text{judge}} - s_{\text{linear}}| \leq 0.10$ clamp at the boundary of `GeminiCompositeJudgeService` is a single-file change. Deploying it requires re-running the ranking-evaluation pipeline on the existing fourteen curricula vitae to produce a freshly clamped result rather than the post-hoc sensitivity check that the chapter currently reports. The sensitivity check (Section 5.6) shows the direction of the resulting shift; production deployment would replace the sensitivity check with a primary endpoint computed under the fix.

Multi-vendor judge ensemble. The principal limitation named in Section 5.8 is the single-vendor judge family. Adding judges from one OpenAI model at its strongest tier and one model from a third vendor family (for example Mistral’s Large tier) would allow inter-judge Cohen’s κ to be measured directly across vendor families and would establish that the reason-quality and ranking-quality results are not artefacts of the Anthropic family chosen for the current evaluation.

Gold-set extension to a per-family inferential sample size. The Layer 5 confidence interval of width approximately 0.23 reflects the n equal to fourteen sample. Extending the curriculum-vitae gold set to at least thirty per role family (Engineering, Data, Product, Design) would support per-family inference and tighten the aggregate confidence interval enough to support a directional verdict in either direction.

Human inter-annotator agreement on a match-quality subset. The match-quality gold ratings are produced by a single judge family and audited by the interested author. An inter-annotator coefficient computed on a stratified subset by an annotator without a stake in the outcome would establish the gold set’s external validity and would complement the judge-self-consistency check reported in Section 5.5.

Continuous-integration and continuous-deployment pipeline. The current deployment is a manual but reproducible script (Section 4.6). A GitHub Actions workflow that builds the image, transfers it to the host, and runs the Compose roll-forward is straightforward to add; the absence of the pipeline is the most concrete operational gap recorded in Section 4.6.

Migration to a multi-instance topology. The single-host deployment described in Section 3.7 has no horizontal redundancy. The migration path to an Elastic Container Service task definition behind an Application Load Balancer is essentially mechanical from the current code base because the layer separation already places the deployable artefact behind a stateless interface; the operational case for the migration would be the first reproducible incident of host unavailability.

A/B production validation of the v6 prompt. The reason-quality result on the v6 prompt was obtained under the judge protocol of Zheng et al. and predicts user satisfaction by literature precedent; it does not measure user satisfaction directly. A v4-versus-v6 A/B comparison under real user click-through and apply rates would close the construct-validity gap acknowledged in Section 5.8.

Reference-free factuality evaluation as a Layer 4 complement. The MiniCheck factuality model described by Tang et al. [9] is integrated into the Infrastructure layer (Section 4.4) as **MLApiFactualityService** but is not evaluated in this thesis. Activating it as a third reason-quality methodology — alongside the existing rubric-based judging and the programmatic checks — would complement the cross-vendor judge ensemble with a reference-free claim-level verifier.

Closing assessment. The thesis delivers a deployed system, a defensible layered evaluation methodology, a statistically significant positive empirical result on reason-text quality, and a directionally non-significant empirical result on ranking quality whose conditions for resolution are stated explicitly. The methodology stands independent of the empirical verdict; the empirical question on ranking quality remains open, with each of the extensions named above proposed as the smallest piece of work whose completion would meaningfully tighten the answer.

Glossary

Bibliography

- [1] S. Danziger, J. Levav, and L. Avnaim-Pesso, “Extraneous factors in judicial decisions,” *Proceedings of the National Academy of Sciences*, vol. 108, no. 17, pp. 6889–6892, 2011, doi: [10.1073/pnas.1018033108](https://doi.org/10.1073/pnas.1018033108).
- [2] R. C. Martin, *Clean Architecture: A Craftsman’s Guide to Software Structure and Design*. Prentice Hall, 2017.
- [3] Court of Justice of the European Union, “CV-Online Latvia SIA v. Melons SIA.” [Online]. Available: <https://curia.europa.eu/juris/document/document.jsf?docid=242047&doclang=EN>
- [4] G. Adomavicius and A. Tuzhilin, “Toward the next generation of recommender systems: A survey of the state-of-the-art and possible extensions,” *IEEE Transactions on Knowledge and Data Engineering*, vol. 17, no. 6, pp. 734–749, 2005, doi: [10.1109/TKDE.2005.99](https://doi.org/10.1109/TKDE.2005.99).
- [5] K. Järvelin and J. Kekäläinen, “Cumulated gain-based evaluation of IR techniques,” *ACM Transactions on Information Systems*, vol. 20, no. 4, pp. 422–446, 2002, doi: [10.1145/582415.582418](https://doi.org/10.1145/582415.582418).
- [6] P. Liang *et al.*, “Holistic Evaluation of Language Models,” *Transactions on Machine Learning Research*, 2023, Accessed: Jun. 03, 2026. [Online]. Available: <https://arxiv.org/abs/2211.09110>
- [7] L. Zheng *et al.*, “Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena,” in *Advances in Neural Information Processing Systems*, 2023. Accessed: Jun. 03, 2026. [Online]. Available: <https://arxiv.org/abs/2306.05685>
- [8] S. Hu and others, “AI Hiring with LLMs: A Context-Aware and Explainable Multi-Agent Framework.” Accessed: Jun. 04, 2026. [Online]. Available: <https://arxiv.org/abs/2504.02870>
- [9] L. Tang, P. Laban, and G. Durrett, “MiniCheck: Efficient Fact-Checking of LLMs on Grounding Documents,” in *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, Association for Computational Linguistics, 2024. [Online]. Available: <https://arxiv.org/abs/2404.10774>

A | Appendix

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aequi doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut.